

BELGIUM CAMPUS ITVERSITY
SEN381

BUGBUSTERS

CIVICCONNECT PROJECT TEAM

MILESTONE 2

Engineering Foundation & Requirements Baseline

PED v2.0 — Engineering Baseline

TEAM MEMBERS

CHRISTIAN JANSE VAN RENSBURG (601840)
MARCO REINERS (578056)
WALDO BLOM (578068)

PROJECT DOCUMENT DETAILS

COURSE Software Engineering 381 (SEN381)

PROJECT CivicConnect

SUBMISSION DATE [30 September 2026]

Controlled engineering documentation • Team BugBusters

Table of Contents

1. Problem, Stakeholder Analysis & Business Value.....	5
1.1 Problem and business need	5
1.2 Stakeholder analysis.....	5
1.2.1 Product owner	5
1.2.2 Development team (Waldo, Christian and Marco – “Bugbusters”).....	5
1.2.3 Requester.....	6
1.2.4 Management.....	6
1.2.5 Staff.....	6
1.2.6 Admin (added during M2).....	7
2. Scope Baseline & Constraints.....	7
2.1 Scope Baseline	7
2.1.1 In Scope	7
2.1.2 Out of Scope	8
2.1.3 Deferred Scope.....	9
2.2 Constraints	10
3. Requirements & Acceptance Criteria.....	11
3.1 Functional Requirements	11
3.2 Non-Functional Requirements	13
4. Architecture-Significant Requirements and Quality Drivers	14
4.1 Identification of Architecture-Significant Requirements.....	14
4.2 Selected CivicConnect ASRs and Quality Drivers.....	14
4.3 Influence of ASRs on M2 Engineering Decisions.....	15
4.4 Quality Requirements Not Treated as Primary ASRs.....	16
5. Architecture Baseline.....	17
5.1 Architecture Context and Evaluation Criteria	17
5.2 Architecture Alternatives Considered.....	18
5.3 Selected Architecture and Rationale.....	20
5.4 Logical Architecture and Responsibilities	20
5.5 Architecture Diagrams.....	21
5.6 Architecture Decision Record and Consequences.....	22
6. Data & Persistence Baseline	23
6.1 Core Data Entities and Relationships.....	23
6.2 Data Model / ERD / Schema.....	30
6.3 Data Ownership and Lifecycle.....	31
6.4 Integrity, Consistency and Concurrency	32
6.5 Scalability, Availability, SPOF & Backup/Recovery	33

7.	Technology Stack Baseline	36
7.1	Technology Requirements and Selection Criteria	36
7.2	Alternatives considered	36
7.3	Selected Technology Stack	36
7.4	Versions and Compatibility Assumptions	37
7.5	Technology Trade-offs	37
7.6	Evidence and traceability	37
8.	Initial Design Decisions	37
8.1	Design decision 1: Decoupling domain logic from external side-effects	37
8.2	Design Decision 2: Centralized state mutation Governance	38
9.	Initial API / Integration Decisions	39
9.1	Interface/Integration background and context	39
9.2	Responsibilities and Information Exchange	39
9.3	Interaction Mechanism	40
9.4	Validation, Errors and Security Boundary	40
10.	Deployment Compatibility	41
10.1	Deployment Direction and Environment Assumptions	41
10.2	Compatibility of Architecture, Data and Technology Decisions	43
10.3	Deployment Configuration, Secrets, Persistence/State and Networking Implications	43
10.4	Deployment Compatibility, Cost, Availability and Operational Risks	45
10.5	Deferred Deployment Decisions and Evidence Required	47
11.	RTM & Traceability Foundation	49
11.1	Evolved RTM	49
11.2	Requester RTM	49
11.3	Staff RTM	49
11.4	Management RTM	50
11.5	Engineering Decision Log	51
12.	Risk & Forward Engineering Considerations	58
12.1	Initial Risk Register	58
	Forward Engineering Considerations	63
13.	Development and Application Evidence	65
13.1	Current Application Status	65
13.2	Repository / Module Structure	65
13.3	Initial Implementation Evidence	66
13.4	Database / Migration Evidence	66
13.5	Testing / Verification Evidence	66
13.6	What is still left (TODOs)	66

14. GitHub Governance	67
14.1 Branches.....	67
14.2 Pull requests (PR's) and Code review process	69
14.3 Issues.....	69
14.4 Milestones	69
14.5 Labels.....	69
14.6 CI / Automated Checks	70
15. Architecture, Technology and Initial Design Baseline.....	71
15.1 Baseline Identification	71
15.2 Baseline synthesis and operational contract.....	71
15.3 Open / Deferred Decisions.....	72
References.....	74
Baseline Sign off.....	76

1. Problem, Stakeholder Analysis & Business Value

1.1 Problem and business need

The problem is that the community focused organization currently manages service requests through different communication platforms: email, telephone calls, WhatsApp messages, spreadsheets and paper-based protocols. This approach creates operational problems such as: requests are frequently duplicated, overlooked, or lost between channels; requesters don't have visibility into the status of their requests; staff struggle to coordinate and prioritise tasks; management does not have enough information about the outstanding, overdue and resolved work; there is no way to keep track of the different requests logged.

The organisation requires a digital platform that can provide a reliable, and easy way to submit, manage, monitor, and report on these different requests. This digital platform aims to solve these operational problems identified above. Stakeholders each have different business needs will interact with the application in different ways, this is further explained in the next section.

1.2 Stakeholder analysis

The project consists of 6 different identified stakeholders, each of them having different needs and ways in which they will interact with the digital platform. Some stakeholders might have conflicts or competing expectations. Below is the breakdown for each stakeholder:

1.2.1 Product owner

This is the person who wants the system in the first place. They are the “community-focused organization” that identified the problem. Their responsibility is to ensure that they clearly communicate their needs and expectations for the system that needs to be developed.

They do not submit actual issues. They review the system during the milestones to ensure that it meets the expectations and aligns with the business needs and functional requirements.

Conflict / competing expectation: The Product owner wants the digital platform to have as much functionality as possible within the fixed budget and schedule. This could clash with Development teams need for a stable scope and enough time to build a quality product that is also maintainable. If the scope is not controlled properly, this could lead to scope creep. That is why scope will be controlled and any changes in scope will not just be accepted, they will first be considered against the baseline scope identified and only in the event that scope change is deemed absolutely necessary, it will be accepted and be properly documented.

1.2.2 Development team (Waldo, Christian and Marco – “Bugbusters”)

This is us (Waldo, Christian and Marco) the “Bugbusters” team who are actually developing the project based on the identified requirements. Our responsibility is to ensure that we clearly understand the business need in order to identify the requirements of the project. But also, to ensure that we build a system that is maintainable, reliable and secure.

We will test the system to ensure that it works correctly. We will not add additional items to the scope before properly considering the implications of doing so, also in the event that the scope changes we will properly document the decision and the change.

Conflict / competing expectation: We must balance the speed at which we develop the application with the quality and maintainability of the end product. With a fixed schedule and fixed tier constraints we must choose quality and sustainability over just implementing something that works and gets the job done.

1.2.3 Requester

This is the person who experiences the actual problem i.e. a fault, security concern, lost property, or any other operational request.

Their responsibility is to provide accurate and clear information about the issue they are experiencing, and to label the issue correctly according to the request category. They should also respond to staff when asked for more information about their issue.

Their core need is to stay updated on their request. They want as much visibility as possible when it comes to their logged request, i.e. knowing when it was received, assigned, in progress and resolved. They want a system that is easy to use and easy to interact with, they don't want something is complex and difficult to use.

Conflict / competing expectation: Requesters want an experience that is simple and quick however the staff and management need sufficient information to be able to resolve the issue that the requester is experiencing. These needs clash and need to be balanced, though the digital platform that will be developed.

1.2.4 Management

Note: Each department manager (e.g., IT Manager, Facilities Manager) is not a separate stakeholder; Management overall is treated as a single stakeholder group.

Management oversees their specific category (i.e. IT sees IT related requests, Facilities Manager sees the requests associated with facility requests) of requests and monitors activity through a dashboard (see FR-301 of the functional requirements section), which thereby provides the visibility of active and closed requests. In order to improve operational efficiency and reduce management workload, staff may self-assign tasks. However, Management still has the final authority to approve, override, or redirect requests as needed.

Their responsibility is to ensure that the issues assigned to their department is resolved within the expected time frame. Also, to authorize release of contact information of the Requesters to the staff only when needed, for example to get further context about an issue when it was not clearly stated (Scope item nr. 9 & out of scope item 2).

Conflict / competing expectation: Management want detailed reporting and accountability of their staff (who did what, when etc.). However, staff want minimal data entry and an interface that is easy and allows for fast data entry. Privacy of the requesters must also be ensured, information must only be provided when absolutely necessary as the requesters value their privacy of their data but at the same time if they are not specific enough about their issue, they will need to be contacted for further clarification.

1.2.5 Staff

Note: Each staff member is not a separate stakeholder; The staff overall is treated as a single stakeholder group.

The staff are the people who are actually assigned to investigate, handle and resolve the service requests made by the requester.

Their primary responsibility is to accept and claim ownership of the issue they are assigned to, including status updates, resolution details and contacting the requester in the event there is clarification needed (Scope item nr. 9 & out of scope item 2).

Their core need is to clearly see the issues that they are assigned to, and to be able to update the status of the requests they are working on. They want an interface that is easy to interact with and that requires minimal clicks and fast loading time (As they are going to be using this system a lot).

Conflict / competing expectation: Staff may resist to use this newly proposed solution in the event that it is slower than the current informal methods (WhatsApp, email, telephone calls etc.). In order to ensure that they will want to use the system, we must make the system as easy as possible to use, while still ensuring management has detailed reporting and accountability of their staff (who did what, when etc.).

1.2.6 Admin (added during M2)

M2 review: During millstone 2 we identified another stakeholder. We refer to this as the admin stakeholder. We identified something that we missed previously; how the staff and managers will be registered. Requesters will be able to self-register, but the application would be insecure if anyone could register as a manager or staff. So, this is where the admin user comes in, they register the managers and managers are mainly responsible for registering staff. However, the admin user will also be able to manage staff across different divisions.

2. Scope Baseline & Constraints

2.1 Scope Baseline

2.1.1 In Scope

These are the core features and things that our application will and must be able to do. These are the things that we are promising to ensure that our system will perform, this is the baseline scope and will be adjusted as the project progresses (note in the event that the scope is adjusted we will properly document these changes):

1. User login, registration and authentication
Requesters will be able to register and log in to access account specific functionality. They will also be able to update their personal information and settings such as if they want receive email push notifications.
This authentication will also help with staff and management functionality as they will also have accounts. This will help with things such as staff being assigned to a specific department and also to a specific management person.
M2 Review: IN PEDv1 we established register for Requesters, Management and Staff. After further consideration we identified in PEDV2 that requesters will be able to self-register. Management will register staff for only their department. A special Admin user will register the management users and will also be able to manage staff across all departments.
2. Requesters submit a new service request
Requesters will be able to submit a new service request containing all the required information.
3. Add images to a request
Requesters will also be able to attach images to a specific request in the event that it is needed. However, images will be limited to a certain file size (still to be determined at a later stage what this established limitation is, as it depends on the database architecture limitations and free tier availability).
4. Controlled request categorization
Requesters will select a request category from a list of available categories. These categories will then be used to make the request available to the relevant department. For example, IT

related issues will only go to the IT department and NOT across the all the departments. This will help to better keep track of what the issue pertains to.

5. Status tracking of a request
The requesters will be able to view the current status of their submitted request as it changes. Staff will be able to change the status of only the issues they are assigned to. Management then also has the dashboard view of the different requests and their status within their department and which staff member is assigned to the specific issue
6. View request history
Requesters will be able to view the history of their previously submitted requests. Managers will also be able to view all the past requests logged within their department. Staff will be able to view their assigned request history.
7. Feedback via email push notification.
As an additional feature that we will implement is the ability for requesters to receive email notifications as their status of their task changes. This feature will be able to be toggled on / off. This is also why we require a user registration with email as we will use this email address to send the push notifications to.
8. Search and filter requests
Staff and Management will be able to search, filter and sort available service requests using relevant criteria.
9. View requester details
Authorized staff will be permitted access to contact details associated with a requester. This is how the staff member will be able to get further clarification on an issue if required. The manager must first approve access to this information for the staff member.
10. Assign / accept responsibility
Authorized staff will be able to accept responsibility for the appropriate unassigned requests. Meaning that they will be able to assign themselves to a specific request only within their department.
11. Management assign and reassignment
Although staff are able to assign themselves to a task within their department, management still has the ability to assign or reassign service requests to the appropriate staff member within their department. This thereby allows managers to only assign task when needed, which allows them to spend more time on other responsibilities they have to attend to such as monitor the performance of their staff via the dashboard.
12. Update status
The assigned staff member of a specific task will have the ability to update the status of the issue as it progresses.
13. Resolve and close request
The assigned staff member will be able to resolve and close the request. Management is able to revert this closer if deemed the issue is not yet resolved.
14. Management dashboard and reporting
Management will be able to view information to open, overdue, resolved and closed requests. Management will be able to analyse request information using the relevant search and filter tools available.
15. Digital platform Mobile accessibility
For our proposed digital platform, some of the screens we will adapt for mobile as this will improve the user experience for the requesters.

2.1.2 Out of Scope

These are explicit decisions that that we will not be moving forward with and are excluded from the baseline scope.

1. AI chatbot
We will not create an AI chatbot for users to communicate with.
It is unlikely that users will make use of this feature. Also, when implementing something like this it can be complex there are certain risks such as people misusing the chatbot for personal reasons (Schuman, 2026). There are security and implementation risks associated that are not worth the potential value that it could add (Schuman, 2026).
2. Internal real time messaging system
Requesters won't have direct communication with staff via the system. Staff will send an email or call the user only when the issue on the request is not clearly stated. The system will have the option to update the status of the issues.
3. Video file upload
Requesters will not be able to attach video files to the service requests. Uploads will be restricted to images only due to storage limitations (images will take less space to store).
4. Voice input
Communication is limited to text only. Dealing with different audio files can get complex and also require a more data storage capacity. This potential feature is not worth the additional storage capacity.
5. Social media login
We will provide one method of registration and that is email along with a password. We will not be integrating Google login, Login via Facebook. This overcomplicates the issue and requires an additional API's that could change or break. This could add to the technical debt of the application.

2.1.3 Deferred Scope

These are recognised possible future things we could implement in future milestones, but we deliberately postpone these issues due to schedule, cost, risk or missing evidence.

1. Auto assign issues to the same task
Ideally, it would be beneficial for the system to automatically group multiple requests to the same underlying issue. For example, a specific website that the community focused organization manages and is in charge of goes down. They might get multiple requests that the website is no longer accessible. So instead of dealing with each individual request we treat it as single issue. This could be advantageous, but this decision is deferred as we would first like to implement the basic functionality first. This is a "nice to have".
2. Portal for requesters to review their interactions with the staff
Adding a reviews portal where requesters could review their interaction with a specific staff member if they had one. This is also another "nice to have".
3. Additional role for across all management departments
An additional user role that will have access to a dashboard to view overall metrics of the system such as how many tasks assigned to what department, how long on average does each department take to resolve specific issues. It would give metrics about the performance of the staff and management overall at a glance (Like an Admin user). So, this is not department specific but a user that has a greater level of access. We defer this decision as we would first like to establish a baseline system and then if there is available time for this feature, we will implement it
4. Extent of the Mobile adaptability
For our proposed digital platform, we would like to have a mobile adaptability for the requesters. But we don't yet know if we want to apply this same mobile adaptability for the Managers or Staff. Further investigation into their needs is required and also a deeper understanding of what

needs to be displayed on the screens is required. For example, the management dashboard will most likely not be made responsive on a mobile screen as this is something that won't be easily achievable.

2.2 Constraints

Constraint	Description	Engineering Implication
Team Size and Resources	The project must be completed by a team of only 3 members.	Because development resources are limited, we must avoid complex architectures that require more overhead. If one team member falls behind, the other two will do the workload, which means code must be readable and documented.
Schedule	The project must progress through 4 formal milestones within 7 weeks.	Time is heavily limited, so scope must be flexible. If a chosen technology is too difficult to learn or if development falls behind the schedule, then we must drop the "could have" and "should have" features rather than to miss a milestone's deadline.
Cost	The team must use free services where possible and identify operational limits.	This heavily restricts our tech stack for M2. We cannot use premium services. We must actively monitor our usage of free limits to ensure we do not hit a paywall.
Scope control	Committed scope must be baselined, uncontrolled scope creep is not acceptable.	Once M1 document is signed off, any new feature ideas cannot be silently added. It must go through a formal Change Request and impact analysis to ensure it does not interfere with our schedule or quality.
Evidence	Attributes must be defined and later supported by measurable evidence and not just claims.	We cannot just simply say the code works; we must prove it. This is implied we must allocate time to write automated test and configure continuous integration pipeline to act as a quality gate before any code merges into the main branch.
Security	Security must be treated as a lifecycle wide engineering responsibility not just a final add-on.	We must implement strict config management from day one. Things like database passwords and API keys must never be committed to our GitHub. This does require us to establish a secure environment strategy.
Technology and team capability	No technology stack is prescribed but decisions must be justified against the constraints and our team capability.	We cannot pick a popular framework just because it looks trendy. If we pick a tech stack that nobody in the team knows there is a learning curve risk and that will threaten our schedule.

3. Requirements & Acceptance Criteria

3.1 Functional Requirements

Priority:

M – Must have.

S – Should have.

C – Could have.

How the Requirement IDs work; The first number is for stakeholder when it comes to FR and category when it comes to NFRs. The second and 3rd number is just for numbering the requirement.

Requirement ID	Group of Feature	Description	Priority	Source/Stakeholder	Acceptance Criteria
FR-101	Submission	The system must allow users to submit a service request using a category mechanism.	M	Requester	When valid details are submitted when the user clicks, save/submit then a unique ID is generated.
FR-102	Tracking	The system must allow users to view the status of requests.	M	Requester	Given that the user inputs their unique ID, then the ticket status is displayed.
FR-103	History	The system should allow authenticated users to view a history of their requests	S	Requester	Given that the user is logged in, when navigating to their profile, then a list of their past request should render.
FR-104	Feedback	The system must provide feedback when a user's request is updated.	M	Requester	When a request is accepted, rejected, updated or completed, the system must generate an appropriate notification for the requester.
FR-105	Authentication & Registration	The system must allow requesters to register an account and authenticate before accessing account-specific functionality.	M	Requester	Given valid registration information, the system must create a requester account. Given valid registered credentials, the system must authenticate the requester. Invalid credentials must be rejected.
FR-106	Attachments	The system should allow requesters to attach supported image files to a service request.	S	Requester	Given a requester creating or submitting a request, when a supported image is attached, the image must be associated with the correct service request and be retrievable by an authorised user.
FR-201	Assignment	The system must allow staff to accept responsibility for a ticket.	M	Staff	Given that an unassigned ticket and the staff user clicks assign, then they must become the record owner.

FR-202	Workflow	The system must allow staff to update request statuses through controlled transitions.	M	Staff	Given a ticket, if updated to a different status then the status state is saved in the database.
FR-203	Staff View	The system must allow the staff to search, filter and sort request using criteria.	M	Staff	Given the staff dashboard, when filtering by criteria, then only tickets related to that criteria are displayed.
FR-204	Resolution	The system must allow staff to record actions, comments or resolution information.	M	Staff	Given that a ticket is being resolved, then the staff member must input a resolution comment before closing it.
FR-205	Request Details	The system must allow authorised staff to view the full permitted details of service requests relevant to them.	M	Staff	Given an authorised staff member opening an available request, the system must display the permitted request information associated with that request.
FR-206	Request Closure	The system must allow authorised staff to resolve and close a service request through the controlled workflow.	M	Staff	Given an authorised staff member resolving a request, when all required resolution information has been provided and a permitted closing transition is performed, the request must be recorded as resolved or closed.
FR-207	Department-Relevant Access	The system must make service requests available to authorised staff according to the request category and relevant staff responsibility or department.	M	Staff	Given a service request assigned to a defined category, authorised staff within the relevant responsibility area must be able to access it, while staff outside the authorised scope must not be able to access it.
FR-301	Dashboard	The system must provide a dashboard to identify open, overdue and closed requests.	M	Management	Given that a manager accesses the dashboard, then a visual metric of active vs closed tickets should render.
FR-302	Auditing	The system should support reporting on request lifecycle history for accountability.	S	Management	Given a completed ticket, then a timestamp audit log of all transitions should be viewable.
FR-303	Assignment & Reassignment	The system must allow authorised management to assign or reassign responsibility for a service request to an appropriate staff member.	M	Management	Given an eligible service request and authorised manager, when a valid staff member is selected and the assignment is confirmed, that staff member must become the recorded assignee and the assignment change must be recorded.
FR-304	Request Analysis	The system must allow authorised management to analyse service-request information using relevant dimensions such as category and status.	M	Management	Given an authorised manager viewing service-request information, when a supported category, status or other approved dimension is selected, the resulting information must correspond to the selected criterion.
FR-305	Contact Information Authorisation	The system must restrict staff access to requester contact information and allow authorised management to control access where additional clarification is required.	M	Management	Requester contact information must not be made available to an unauthorised staff member. Where the required access has been authorised for a request, the permitted information must become available only to the appropriate authorised staff member.

3.2 Non-Functional Requirements

Requirement ID	Category	Description	Measurable Acceptance criteria	Justification
NFR-101	Security	The system must enforce authentication and data protection.	Passwords must be hashed. Data in transit must use https. Role-Based Access Control must prevent requesters from accessing staff endpoints. (measurable through test gates, binary pass/fail)	“Security is a lifecycle wide responsibility not a final add-on”, Because CivicConnect handles sensitive community data such as locations and contact details, failing to secure trust early creates privacy risks.
NFR-201	Performance	The platform must respond efficiently to user inputs to prevent duplicate submissions.	95% of all standard API read and write requests must compile in less than 800 milliseconds under normal system load.	If the system responds slow, then users will click “submit/save” multiple times, leading to duplicated tickets. By setting a millisecond limit, gives the team a measurable goal.
NFR-301	Constraints	The architecture must operate for free (open source) or within low cost.	The production environment must run on free-tier services without exceeding monthly quotas.	Cost is an engineering constraint in this Project.
NFR-401	Compatibility	Given that the decision is made that the system needs to operate on multiple different platforms -The public portal must be usable on modern desktop and mobile browsers. Please note this decision will only be made in milestone 2.	The UI must pass responsive visual testing on the newest versions of Chrome, Safari, Edge and Firefox without horizontal scrolling on mobile.	Since native mobile applications are out of scope, the web interface is the only way citizens can submit a new service request. Requesters will most likely report issues on the go, using their mobile browsers, making responsiveness a crucial attribute.
NFR-501	Reliability	The system must handle erroneous inputs without crashing.	100% of the form submissions must have backend validation. The system must return human readable http error codes instead of stack traces.	If the system crashes silently on bad input, tickets are lost. Validation ensures data integrity and provides feedback to the user.

4. Architecture-Significant Requirements and Quality Drivers

4.1 Identification of Architecture-Significant Requirements

Not every CivicConnect requirement has the same influence on the architecture. For Milestone 2, the team reviewed the existing functional requirements, non-functional requirements, stakeholder needs, project constraints, risks and forward engineering considerations to identify the requirements and quality attributes that materially affect the structure and engineering direction of the system. These are treated as Architecture-Significant Requirements (ASRs) because failing to address them early could require substantial redesign, introduce significant project risk, or prevent important stakeholder and project requirements from being achieved.

The identified ASRs are used as decision criteria when evaluating the CivicConnect architecture, data and persistence approach, technology stack and initial design.

Measurable expectations are retained or defined where appropriate so that the effect of these decisions can later be verified through implementation and testing. The team has deliberately selected only the drivers that materially influence the system rather than treating every non-functional requirement as an ASR.

4.2 Selected CivicConnect ASRs and Quality Drivers

ID	ASR / Quality Driver	Project Evidence / Source	Measurable / Testable Expectation	Why It Is Architecture-Significant
ASR-01	Security, Authorisation and Controlled Access	NFR-101; FR-105 Authentication & Registration; FR-207 Department-Relevant Access; FR-305 Contact Information Authorisation; new M2 Admin stakeholder; RSK-003 Unauthorised access to requester information.	Passwords must be securely hashed; protected communication must use HTTPS; unauthorised users must be denied protected functionality; role and department restrictions must be enforced server-side; requester contact information must only be accessible where the required authorisation exists.	CivicConnect contains Requester, Staff, Management and Admin roles with different levels of access. Requests are department-restricted and some requester information requires additional Management approval. Security therefore cannot exist only in the user interface; authentication and authorisation must influence application boundaries, request handling, business logic and persistence relationships.
ASR-02	Data Integrity, Lifecycle Traceability and Reliability	FR-202 Workflow; FR-204 Resolution; FR-302 Auditing; FR-303 Assignment & Reassignment; NFR-501 Reliability; RSK-008 Email notification dependency/failure; RSK-010 Loss or corruption of platform data.	Valid request, assignment and status changes must be persisted correctly; invalid input must be rejected through backend validation; required lifecycle changes must be traceable; failure of a secondary function such as email notification must not invalidate or lose the underlying request update.	A central business purpose of CivicConnect is to replace fragmented processes in which requests can be lost, duplicated or poorly tracked. Request state, assignment and audit information must therefore remain consistent across the application. This materially influences the database model, transaction boundaries, validation, audit/history storage and separation between core request processing and secondary functions such as notifications.
ASR-03	Maintainability, Modifiability and Testability	Three-person team constraint; limited milestone schedule; RSK-001 Schedule overrun; RSK-009 Team-member availability/knowledge concentration; evolving requirements across M2-M4.	Core responsibilities must be separated sufficiently for components/modules to be understood, changed and tested independently where practical. Core business logic should be testable without requiring the complete user interface or external services to execute. Significant code must remain understandable and reviewable by more than one team member.	The application must continue evolving across later milestones while being developed by only three team members. Excessive coupling or unnecessary architectural complexity would increase development, testing and knowledge-transfer effort. This driver therefore influences module boundaries, separation of concerns, dependency management and the level of architectural complexity that is acceptable.
ASR-04	Performance and Interaction Responsiveness	NFR-201 Performance; Staff stakeholder need for fast interaction and minimal delay; Management search/filter/dashboard requirements.	The existing NFR target remains that 95% of standard API read and write requests complete within 800 milliseconds under normal system load.	Staff are expected to use the system regularly for request handling, while Requesters need efficient submission and tracking. Performance therefore influences request-processing overhead, database queries and indexing,

ID	ASR / Quality Driver	Project Evidence / Source	Measurable / Testable Expectation	Why It Is Architecture-Significant
			Repeated user actions must not be required because normal operations respond too slowly.	search/filter implementation and the amount of unnecessary communication introduced between application components.
ASR-05	Cost-Effective Deployability and Resource Efficiency	NFR-301; three-person team; free/low-cost project constraint; RSK-005 Unsuitable technology/deployment platform; RSK-006 Free-tier resource/service limitations; image-upload and email features.	CivicConnect must be executable in the team's development environment and must remain deployable using a free or sufficiently low-cost environment appropriate to the educational project. Hosting, database, storage and supporting-service limitations must be identified before final commitment, and secrets/configuration must remain external to source control.	Cost and deployment limits constrain the amount of infrastructure and operational complexity the team can responsibly support. Architecture, persistence, image storage, notifications and deployment must therefore avoid requiring unjustified infrastructure or paid services and remain practical for a three-person team to configure, test and maintain.

4.3 Influence of ASRs on M2 Engineering Decisions

ASR	Architecture Influence	Data / Persistence Influence	Technology Influence	Initial Design Influence
ASR-01 Security, Authorisation & Controlled Access	Requires clear security boundaries and consistent enforcement of authentication and authorisation throughout the server-side application. Architecture alternatives must support centralised and reusable access-control rules rather than relying only on UI restrictions.	User roles, departments, staff assignments, requester ownership and controlled contact-information access must be represented in the data model. Access relationships must support both role-based and department-based restrictions.	Authentication technology must support secure credential/token or session handling, password hashing and server-side authorisation. The chosen web framework and deployment platform must support secure configuration and secrets management.	Controllers/routes and business operations must check authorisation before protected actions are performed. Reusable access-control logic should be preferred over duplicating permission rules throughout the application.
ASR-02 Data Integrity, Traceability & Reliability	Architecture must maintain a clear responsibility for request workflow and persistence so that request state does not become inconsistent across components. Secondary operations such as notification should not determine whether the underlying request update succeeds.	A structured persistence model must represent requests, status/history, assignments, departments, users and audit information. Integrity constraints, validation and transaction support must be considered when finalising the persistence approach.	The database technology must provide reliable persistent storage and appropriate integrity/transaction capabilities. This requirement will be considered when finalising the current PostgreSQL direction.	Workflow rules and validation should not be duplicated across unrelated UI/controllers. Operations that logically belong together must be handled consistently, while notification/audit responsibilities should not unnecessarily corrupt or control the core request transaction.
ASR-03 Maintainability, Modifiability & Testability	Architecture alternatives must provide meaningful separation of responsibilities without introducing unnecessary distributed complexity. Clear internal module/layer boundaries are preferable to a structure where unrelated concerns become tightly coupled.	Persistence access should be separated sufficiently from presentation and request-routing concerns so that changes to storage logic do not unnecessarily affect the complete application.	Technology choices should favour tools the team can realistically understand, test and maintain within the available schedule. The current JavaScript/Node.js/Express direction and Jest testing proposal must therefore be evaluated partly against team capability and maintainability.	MVC/separation of concerns, cohesive components and limited coupling should guide design. Core business behaviour should be testable independently where practical.
ASR-04 Performance & Interaction Responsiveness	Architecture should avoid unnecessary processing and communication overhead for ordinary request operations. Introducing distributed/network boundaries must therefore have a clear benefit rather than	Frequently used queries, authentication lookups, request filtering and dashboard operations will require suitable database relationships, indexing and query design.	The selected runtime, framework and database must be capable of meeting the existing response target for the expected project workload.	Request submission should prevent unnecessary duplicate actions; search/filter operations should use efficient queries rather than processing unnecessarily large result sets in the presentation layer.

ASR	Architecture Influence	Data / Persistence Influence	Technology Influence	Initial Design Influence
	being added only for architectural complexity.			
ASR-05 Cost-Effective Deployability & Resource Efficiency	Architecture alternatives must remain proportionate to the size of CivicConnect and practical to deploy and operate with limited resources. Infrastructure-heavy or distributed approaches require stronger justification.	Database and image-storage choices must consider free/low-cost storage limits, persistence availability and recovery implications.	Railway and Render remain deployment candidates and should be compared against cost/free-tier availability, PostgreSQL compatibility, configuration/secrets support, operational simplicity and team capability. Local execution remains useful for development and demonstration, while Docker should only be introduced if it provides sufficient project value to justify the additional complexity.	Configuration should remain externalised from source code. Image size/type controls and failure behaviour for supporting services should be designed with resource limitations in mind.

4.4 Quality Requirements Not Treated as Primary ASRs

The team also reviewed the remaining non-functional and usability concerns but did not automatically classify each one as a primary ASR. In particular, NFR-401 browser/mobile compatibility remains an important quality requirement, but its current effect is concentrated mainly on the Requester-facing presentation and interaction design rather than determining the fundamental backend or persistence architecture. The M2 decision already establishes that Requester-facing functionality will support mobile browser use while the extent of Staff and Management mobile responsiveness remains deliberately deferred. NFR-401 will therefore remain a supporting quality driver and will primarily influence UI design, browser compatibility testing and responsive-layout decisions.

Similarly, detailed CI quality gates and later production observability remain important engineering concerns but are not treated as independent architecture drivers at this stage. They will continue to influence testability, deployment and later M3 quality/release-readiness work rather than being added as separate ASRs merely to expand the list

5. Architecture Baseline

5.1 Architecture Context and Evaluation Criteria

CivicConnect requires an architecture that supports the current functional requirements and the Architecture-Significant Requirements identified in Section 4, while remaining practical for a three-person development team working within a limited schedule and free or low-cost resource constraints. The architecture must therefore provide sufficient separation of responsibilities, support the required quality attributes, and avoid complexity that is not justified by the project.

The architecture alternatives will be evaluated against the following project-specific criteria:

- **Security and controlled access (ASR-01):** The architecture must support consistent authentication and authorisation, including role-based and department-based access restrictions and controlled access to requester information.
- **Data integrity and traceability (ASR-02):** The architecture must support reliable request lifecycle, assignment, status and audit information without introducing unnecessary consistency problems.
- **Maintainability and testability (ASR-03):** Responsibilities should be sufficiently separated so that parts of the system can be understood, changed and tested without excessive coupling.
- **Performance (ASR-04):** The architecture should avoid unnecessary processing and communication overhead that could negatively affect normal request, search and dashboard operations.
- **Cost and deployability (ASR-05):** The architecture must remain practical to run and deploy using free or low-cost infrastructure with limited operational overhead.
- **Team capability and schedule:** The architecture must be realistic for three students to implement, test, document and maintain within the available milestones.
- **Proportional complexity:** Additional architectural complexity must provide a clear CivicConnect-specific benefit. Distributed components or additional infrastructure should not be introduced unless they are justified by the project requirements, risks or constraints.

The Master Project Brief requires architecture and technology decisions to be justified against the project requirements and constraints rather than selected according to preference alone. SEN381 Master Project Brief The Milestone 2 brief also states that a more distributed architecture is not automatically more advanced and that additional complexity must be justified by project evidence. SEN381_CivicConnect_Milestone_2

These criteria will be used in the following section to compare the realistic architecture alternatives before the final CivicConnect architecture is selected.

5.2 Architecture Alternatives Considered

Three realistic architecture alternatives were considered for CivicConnect: a Traditional Layered Monolith, a Modular Monolith, and a Microservices / Distributed Architecture. The alternatives were compared against the project-specific criteria identified in Section 5.1. The purpose of the comparison is not to select the most complex architecture, but to identify the approach that best satisfies the CivicConnect ASRs, project constraints and team capability.

Project-Fit Colour Scale

Best Fit	Good Fit	Acceptable Fit	Weak Fit / Added Complexity	Too Complex / Not Suitable
Strongest match for CivicConnect and its constraints.	Fits the project well with only minor limitations.	Usable, but has notable trade-offs for this project.	Possible, but introduces unnecessary limitations or complexity.	Poor fit for the current scope, team size or constraints.

Comparison of Architecture Alternatives

Evaluation Criteria	Traditional Layered Monolith	Modular Monolith	Microservices / Distributed Architecture
Security and controlled access	Good Fit. Centralised authentication and authorisation are relatively simple to enforce within one application.	Best Fit. Centralised security can be maintained while access-related responsibilities are separated into clear internal modules.	Acceptable Fit. Can provide strong service boundaries, but security must be managed across multiple services and network interfaces.
Data integrity and traceability	Good Fit. A single application and database make transactions and lifecycle consistency easier to manage.	Best Fit. Retains simple transaction handling while allowing request, workflow and audit responsibilities to be separated internally.	Weak Fit / Added Complexity. Data and transactions may need to span multiple services or databases, adding consistency and coordination complexity.
Maintainability and modifiability	Acceptable Fit. Can be maintainable initially, but responsibilities may become tightly coupled as the application grows.	Best Fit. Functional areas can be separated into modules while remaining within one application and one deployment boundary.	Weak Fit / Added Complexity. Provides strong service separation, but adds multiple codebases, interfaces and operational responsibilities that are heavy for this team.
Testability	Good Fit. Layers can be tested independently if separation is maintained correctly.	Best Fit. Clear modules and internal boundaries support focused unit and integration testing without distributed-system overhead.	Acceptable Fit. Services can be tested independently, but substantially more integration and end-to-end testing is required.
Performance and responsiveness	Good Fit. Internal operations use direct in-process communication with little additional overhead.	Best Fit. Internal module communication remains in-process while preserving clearer responsibility boundaries.	Weak Fit / Added Complexity. Network communication between services introduces additional latency and more failure points.
Deployment simplicity	Best Fit. One application is straightforward to configure and deploy.	Best Fit. Retains one primary deployment unit despite clearer internal structure.	Too Complex / Not Suitable. Several services may need to be configured, deployed, connected and monitored separately.
Free / low-cost operation	Best Fit. Requires relatively little infrastructure and is practical for free or low-cost hosting.	Best Fit. Provides additional structure without requiring multiple hosting environments or distributed infrastructure.	Weak Fit / Added Complexity. Multiple services, databases or supporting infrastructure may increase resource use and cost.

Evaluation Criteria	Traditional Layered Monolith	Modular Monolith	Microservices / Distributed Architecture
Three-person team capability	Good Fit. Relatively simple for a small team to understand and maintain.	Best Fit. Provides useful separation of responsibilities without introducing excessive operational complexity for three team members.	Too Complex / Not Suitable. Requires additional development, integration, deployment and maintenance effort that is not realistic for the current team size.
Limited milestone schedule	Best Fit. Can be implemented quickly, although poor separation may create later maintenance problems.	Best Fit. Adds useful structure while remaining achievable within the available milestone schedule.	Too Complex / Not Suitable. Significant time would be spent on service infrastructure and integration instead of CivicConnect functionality.
Future growth and change	Acceptable Fit. The application can grow, but larger changes may become harder if responsibilities become tightly coupled.	Best Fit. Modules can evolve independently within the application and provide a clear path for later restructuring if needed.	Good Fit. Offers strong theoretical scalability and independent deployment, although this capability exceeds current project needs.
Overall complexity relative to project needs	Good Fit. Simple and proportionate, but may provide too little internal separation as CivicConnect develops.	Best Fit. Provides useful architectural structure while keeping deployment and operations simple enough for the project.	Too Complex / Not Suitable. The additional service, network and operational complexity is not justified by the current scope, team or deployment constraints.

Comparison Outcome

The Traditional Layered Monolith is simple to build, test and deploy, but it may become harder to maintain as CivicConnect grows and responsibilities become more tightly coupled. In contrast, a Microservices / Distributed Architecture provides stronger separation but introduces unnecessary deployment, integration, monitoring and data-consistency complexity for a three-person team working within a limited schedule and low-cost environment.

For these reasons, the **Modular Monolith** was selected as the most appropriate architecture for CivicConnect. It provides a balance between the simplicity of a traditional monolith and the separation of responsibilities normally associated with more distributed architectures. CivicConnect can remain a single deployable application while internally separating major responsibilities into modules such as:

- **Authentication and User Management**
- **Request Management**
- **Assignment and Workflow**
- **Audit and Request History**
- **Notifications**
- **Management Reporting and Dashboard**

These modules can operate through controlled internal boundaries while sharing the same application and persistence environment. This allows CivicConnect to maintain straightforward deployment, transaction handling and performance while improving maintainability, testability and separation of concerns. If the system were to grow substantially in the future, these clearer internal boundaries would also make later architectural changes easier than if the system were developed as a tightly coupled monolith

5.3 Selected Architecture and Rationale

Based on the comparison in Section 5.2, the **Modular Monolith** was selected as the architecture for CivicConnect. It provides the best balance between clear separation of responsibilities and the implementation simplicity required by the current project.

CivicConnect will remain a **single deployable application**, while its major responsibilities will be separated into clear internal modules. The initial responsibility areas are:

- Authentication and User Management
- Request Management
- Assignment and Workflow
- Audit and Request History
- Notification Management
- Search, Reporting and Dashboard

These responsibility areas align with the current CivicConnect requirements for authentication, request submission and tracking, controlled workflow and assignment, auditing, notifications, search/filtering and management reporting.

The modular monolith allows these areas to remain logically separated while still sharing the same application and persistence environment. This supports simpler deployment, consistent security enforcement, straightforward transaction handling and focused testing without introducing the additional infrastructure and coordination required by independently deployed services.

The main trade-off is that the modules cannot be deployed or scaled independently. This is accepted because the current CivicConnect requirements do not justify the added complexity of a distributed architecture. If the system grows significantly in future, the clearer internal module boundaries would provide a stronger foundation for later architectural change.

5.4 Logical Architecture and Responsibilities

Within the selected Modular Monolith, CivicConnect will use clear internal responsibility boundaries so that presentation, request handling, business rules and persistence concerns do not become mixed together. The aim is to keep the application cohesive while preventing one part of the system from taking responsibility for unrelated behaviour.

The logical responsibility flow will follow these principles:

- **Presentation responsibilities** handle what each stakeholder sees and the input they provide. User-interface components should display information and collect input, but should not contain core workflow, authorisation or persistence rules.
- **Request-handling responsibilities** receive user actions and direct them to the appropriate application behaviour. They should coordinate the request rather than contain all business logic themselves.
- **Business-rule responsibilities** contain the rules that control CivicConnect behaviour, such as who may access information, how requests move through their lifecycle, and how assignment or closure decisions are handled.

- **Persistence responsibilities** handle the storage and retrieval of application data. Database operations should remain separated from presentation logic and should support the integrity and traceability requirements already identified in the PED.
- **Supporting responsibilities**, such as notifications, auditing and reporting, should interact with the core request workflow without unnecessarily controlling or duplicating it.

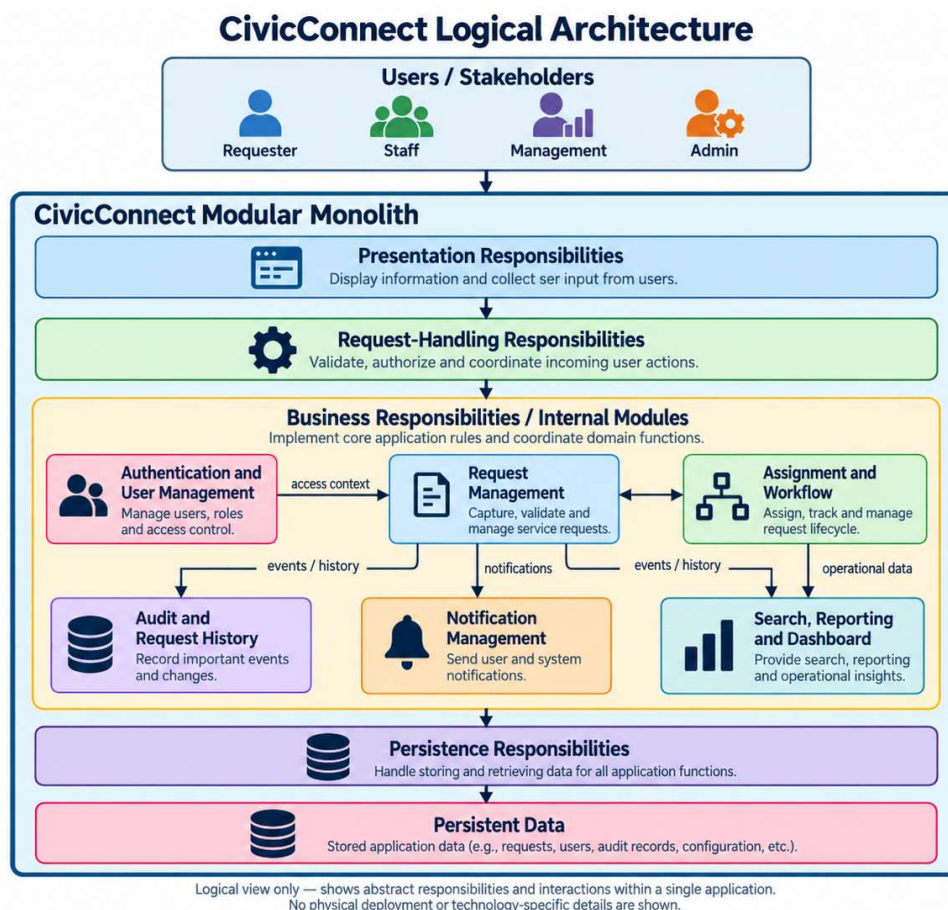
Interaction between internal modules should occur through clearly defined responsibilities. A module should use another module's functionality where required rather than reimplementing the same rule in several places. For example, request-processing functionality should rely on the established authentication and access-control responsibilities when determining whether a user may perform an action, while significant workflow changes should be passed to the audit/history responsibility rather than recorded independently in several parts of the system.

This logical separation supports the maintainability and testability goals identified in ASR-03 while still keeping CivicConnect within one application boundary. It also supports the project's requirement for controlled request lifecycle information and accountability without introducing separate network services or additional deployment boundaries. The project brief requires architecture and design decisions to remain proportionate and traceable to actual project needs rather than adding unnecessary complexity.

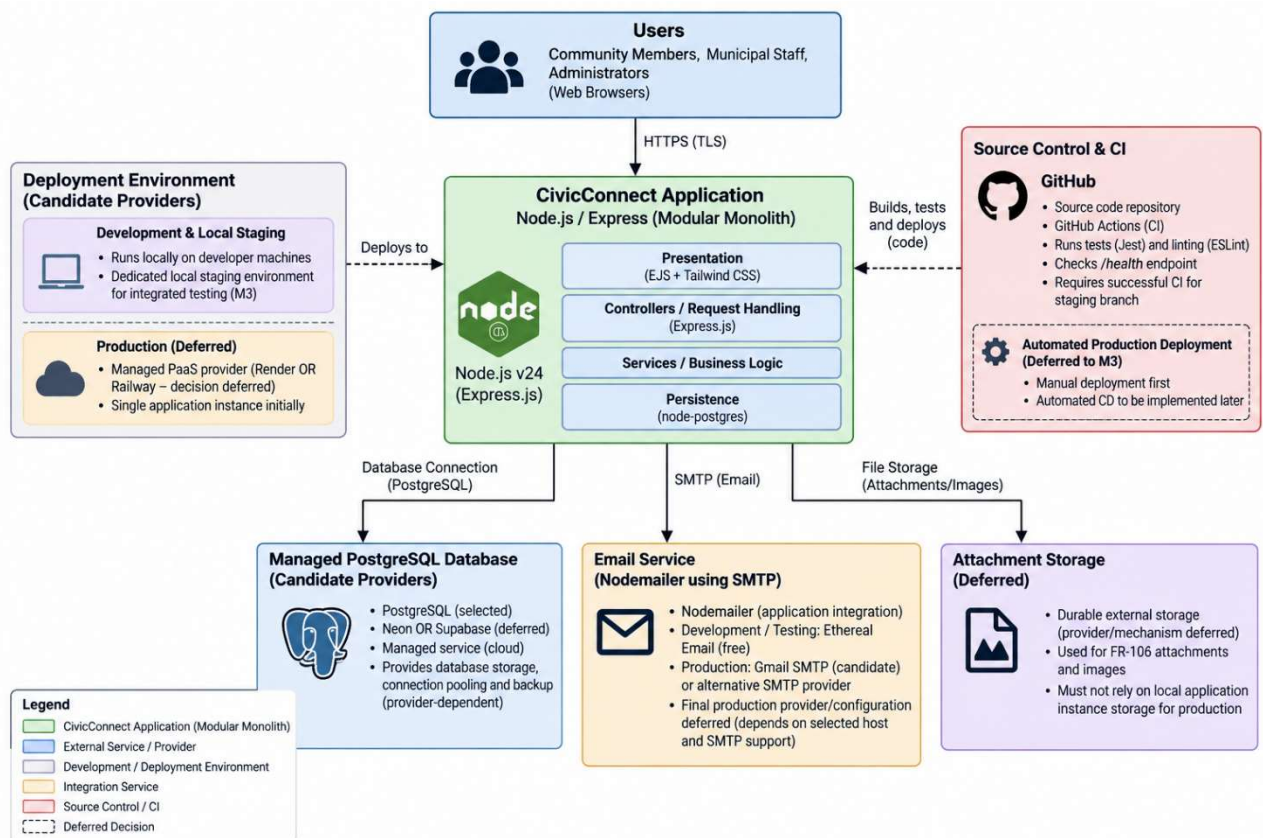
5.5 Architecture Diagrams

The architecture diagrams below show CivicConnect from two different perspectives. The first diagram shows the **logical structure and interaction of responsibilities within the Modular Monolith**, while the second shows the **physical deployment view**. Keeping these views separate avoids confusing internal modules with independently deployed services, which is explicitly required by the Milestone 2 brief.

Logical Architecture Diagram



Physical Deployment Diagrams



5.6 Architecture Decision Record and Consequences

The selection of the **Modular Monolith** as the CivicConnect architecture is recorded in a dedicated Architecture Decision Record (ADR). The ADR captures the alternatives considered, the ASRs and project constraints that influenced the decision, the rationale for selecting the Modular Monolith, and the trade-offs and consequences that follow from this choice. The ADR is maintained as a controlled project artefact in the GitHub repository and linked to the relevant PED sections and supporting evidence.

Decision Consequences

The main consequences of selecting the Modular Monolith are:

- CivicConnect will be developed and deployed as a single application, reducing deployment and operational complexity.
- The application will be separated into clear internal modules, supporting maintainability, testability and separation of responsibilities.
- Communication between modules remains within the same application process, avoiding unnecessary network overhead and distributed-system failure concerns.
- Request workflow, audit and persistence operations can remain within a common application and database boundary, simplifying transaction handling and data consistency.
- The architecture remains practical for the current three-person team, limited milestone schedule and low-cost deployment constraints.

- Internal module boundaries must be maintained during implementation; otherwise, the application could gradually become tightly coupled.
- Individual modules cannot be deployed or scaled independently.
- A failure in the main application process may affect the entire system because all modules run within the same application.
- If CivicConnect grows significantly in future, selected modules may need to be restructured or extracted. The defined internal boundaries should make this easier than if the system were implemented as a tightly coupled monolith.

If later project evidence requires a significant change to the architecture baseline, the ADR and affected PED content will be updated through the project's controlled change process.

6. Data & Persistence Baseline

The following sections describe the tables that make up our database. For each table, the description states its purpose and the requirements it satisfies, names the tables it refers to (the relationships) and the tables that refer to it by means of foreign keys, lists its columns, and describes its indexes and constraints.

All primary keys are UUIDs generated by the database.

6.1 Core Data Entities and Relationships

6.1.1 The users Table

The users table is used for authentication and authorisation of all user types (requesters, staff, managers and administrators), for user profile management, and for tracking user activity.

Satisfies FR-105 (Authentication & Registration), FR-207 (Department-Relevant Access) and NFR-101 (Security).

The users table does not refer to any other table. It is referred to by the departments table (one-to-one, via manager_id), the staff_assignments table (one-to-many, via staff_id and assigned_by), the service_requests table (one-to-many, via requester_id, assigned_staff_id, assigned_by and contact_approved_by), the request_attachments table (one-to-many, via uploaded_by), the request_status_history table (one-to-many, via changed_by), the request_assignment_history table (one-to-many, via assigned_to_staff_id and assigned_by_id), the request_comments table (one-to-many, via commented_by) and the admin_audit_log table (one-to-many, via admin_id). Each of those tables uses the users table for one or more roles, such as requester, assignee, assigner, manager or the user who performed an action.

Columns

- user_id: A primary key used to uniquely identify each user.
- email: The user's email address, used to log in. Must be unique.
- password_hash: The hash of the user's password. The password itself is never stored.
- first_name: The user's first name.
- last_name: The user's last name.
- phone: The user's telephone number. Optional.
- user_type: The role of the user. Can be one of: REQUESTER, STAFF, MANAGER, ADMIN.
- is_active: Indicates whether the user is active. Setting this to FALSE serves as an alternative to deleting a user. Defaults to TRUE.
- email_notifications_enabled: Indicates whether the user receives email notifications. Defaults to TRUE.

- `created_at`: When the user was created.
- `updated_at`: When the row was most recently updated.
- `last_login_at`: When the user last logged in.

Indexes and constraints

- Primary key on `user_id`.
- Unique on email, used for fast authentication lookups.

6.1.2 The `admin_audit_log` Table

The `admin_audit_log` table records administrative actions, tracks changes made to the system, and maintains accountability for user and system management. It is an independent audit trail and is not linked to individual service requests.

Satisfies FR-302 (Auditing) and NFR-101 (Security).

The `admin_audit_log` table refers to the `users` table (many-to-one, via `admin_id`). It is not referred to by any other table.

Columns

- `audit_id`: A primary key used to uniquely identify each audit record.
- `admin_id`: A foreign key identifying the administrator who performed the action, in the `users` table.
- `action_type`: The action that was performed. Can be one of: `USER_CREATED`, `USER_MODIFIED`, `USER_DEACTIVATED`, `DEPARTMENT_CREATED`, `DEPARTMENT_MODIFIED`, `STAFF_ASSIGNED`, `STAFF_REMOVED`, `MANAGER_CREATED`, `MANAGER_REMOVED`, `CATEGORY_CREATED`, `CATEGORY_MODIFIED`.
- `target_type`: The kind of record the action was performed on. Can be one of: `USER`, `DEPARTMENT`, `CATEGORY`, `ASSIGNMENT`.
- `target_id`: The identifier of the record the action was performed on. This is not a foreign key, because it may point to a row in any of the tables named by `target_type`.
- `changed_data`: A JSON document describing what was changed.
- `created_at`: When the action was recorded.

Indexes and constraints

- Primary key on `audit_id`.
- No secondary indexes are defined in the current script.

6.1.3 The `departments` Table

The `departments` table organises service request categories by department. It allows staff to be assigned to departments and supports department-specific dashboards and reporting.

Satisfies FR-207 (Department-Relevant Access), FR-301 (Management Dashboard) and FR-304 (Request Analysis).

The `departments` table refers to the `users` table (one-to-one, via `manager_id`) and is referred to by the `staff_assignments` table (one-to-many, via `department_id`) and the `request_categories` table (one-to-many, via `department_id`).

Columns

- `department_id`: A primary key used to uniquely identify each department.
- `name`: The name of the department. Must be unique.
- `description`: A description of the department.

- `manager_id`: A foreign key identifying the manager of the department, in the users table. Every department has exactly one manager, and a user can manage at most one department, which makes this a one-to-one relationship.
- `is_active`: Indicates whether the department is active. Setting this to FALSE serves as an alternative to deleting a department.
- `created_at`: When the row was created.
- `updated_at`: When the row was most recently updated.

Indexes and constraints

- Primary key on `department_id`.
- Unique on `name`.
- Unique, not-null constraint on `manager_id`. This enforces the one-to-one relationship with users and also supports manager lookups for dashboard filtering and department assignment validation.

6.1.4 The `staff_assignments` Table

The `staff_assignments` table records which staff members belong to which departments and enforces department-based access control on requests. It keeps a history of moves, removals and reassignments, but a staff member can have only one active assignment at a time.

Satisfies FR-207 (Department-Relevant Access) and FR-302 (Auditing).

The `staff_assignments` table refers to the users table (many-to-one, via `staff_id` and `assigned_by`) and the departments table (many-to-one, via `department_id`). It is not referred to by any other table.

Columns

- `staff_assignment_id`: A primary key used to uniquely identify each staff assignment.
- `staff_id`: A foreign key identifying the staff member, in the users table. A user can have many assignments over time.
- `department_id`: A foreign key identifying the department the staff member is assigned to, in the departments table.
- `assigned_by`: A foreign key identifying the user who made the assignment, in the users table.
- `is_active`: Indicates whether this is the staff member's current assignment. When a staff member moves or is removed, the row is kept and this column is set to FALSE. Defaults to TRUE.
- `assigned_at`: When the staff member was assigned to the department.
- `removed_at`: When the staff member was removed from the department. NULL while the assignment is active.

Indexes and constraints

- Primary key on `staff_assignment_id`.
- Partial unique index on `staff_id` where `is_active` is TRUE. This enforces that a staff member has only one active department at a time, and is used to validate access permissions.
- Unique index on (`staff_id`, `department_id`), which prevents the same staff member being assigned to the same department more than once.

6.1.5 The `service_requests` Table

The `service_requests` table stores service requests. It is the central record for all request-related operations, tracking status progression and managing assignment.

Satisfies FR-101 (Submission), FR-102 (Status Tracking), FR-201 (Assignment), FR-202 (Workflow), FR-204 (Resolution), FR-206 (Closure), FR-301 (Dashboard), FR-302 (Auditing) and FR-305 (Contact Information Authorization).

The service_requests table refers to the users table (many-to-one, via requester_id, assigned_staff_id, assigned_by and contact_approved_by) and the request_categories table (many-to-one, via category_id). It is referred to by the request_attachments, request_assignment_history, request_comments and request_status_history tables (each one-to-many, via request_id). Deleting a service request deletes its rows in all four of those tables.

Columns

- request_id: A primary key used to uniquely identify each service request.
- request_number: A unique, human-readable reference number for the request.
- requester_id: A foreign key identifying the requester who submitted the request, in the users table.
- category_id: A foreign key identifying the category of the request, in the request_categories table. The category determines which department handles the request.
- title: A short title for the request.
- description: A full description of the request.
- location: Where the issue is located. Optional.
- status: The current status of the request. Can be one of: SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, REJECTED. Defaults to SUBMITTED.
- priority: The priority of the request. Can be one of: LOW, MEDIUM, HIGH, URGENT. Defaults to MEDIUM.
- assigned_staff_id: A foreign key identifying the staff member currently assigned to the request, in the users table. NULL until the request is assigned. Earlier assignments are kept in request_assignment_history.
- assigned_at: When the current assignment was made.
- assigned_by: A foreign key identifying the manager or staff member who made the current assignment, in the users table.
- contact_information_approved: Indicates whether access to the requester's contact information has been approved. Defaults to FALSE.
- contact_approved_by: A foreign key identifying the manager who approved access to the contact information, in the users table.
- contact_approved_at: When access to the contact information was approved.
- created_at: When the request was submitted.
- updated_at: When the row was most recently updated.
- expected_completion_at: When the request is expected to be completed, used to monitor how long a request should take.
- resolved_at: When the request was resolved.
- closed_at: When the request was closed.
- overdue_flag: Indicates whether the request is overdue.
-

Indexes and constraints

- Primary key on request_id.
- Unique on request_number.
- Index on requester_id, used to find requests by requester.
- Index on assigned_staff_id, used to find staff workload.
- Index on category_id, used for departmental filtering.
- Index on status, used for dashboard filtering.
- Index on created_at, used for date-based queries.
- Index on expected_completion_at, used to monitor expected completion times.
- Index on overdue_flag, used to identify overdue requests.

6.1.6 The request_assignment_history Table

The request_assignment_history table maintains a complete audit trail of assignment changes, recording who assigned a request to whom, and when. This supports accountability and management reporting.

This table is used only to record when someone is removed from, or re-assigned on, a service request. The active assignment is held on the service_requests table.

Satisfies FR-302 (Auditing) and FR-303 (Assignment & Reassignment).

The request_assignment_history table refers to the service_requests table (many-to-one, via request_id) and the users table (many-to-one, via assigned_to_staff_id and assigned_by_id). It is not referred to by any other table.

Columns

- assignment_history_id: A primary key used to uniquely identify each assignment history record.
- request_id: A foreign key identifying the service request the record belongs to. Deleting the request deletes its assignment history.
- assigned_to_staff_id: A foreign key identifying the staff member who was assigned, in the users table. Optional.
- assigned_by_id: A foreign key identifying the user who made the assignment, in the users table.
- assigned_at: When the assignment was made.
- removed_at: When the staff member was removed from, or replaced on, the request. NULL while the assignment is current.

Indexes and constraints

- Primary key on assignment_history_id.
- Index on request_id, used to retrieve the assignment history of a specific request.
- Index on assigned_at, used for timeline-based queries.

6.1.7 The request_attachments Table

The request_attachments table stores references to files uploaded against service requests, manages file metadata, and tracks who uploaded each file. Files themselves are not stored in the database. Only the location of the file and its metadata are kept, which is recommended above directly storing images in the database (Douslimi, 2021). The problem with storing images directly in the database is that it does not scale efficiently leading to performance bottlenecks, it also requires more backend server performance (Douslimi, 2021). Since we want to limit the cost as much as possible, we stick to only storing a reference to where it will be stored. We have not yet decided where exactly the images will be stored but we are considering a cloud service such as Google Drive or One Drive, other solutions will be explored in M3, and in M3 the final decision will be made as to where we will store the actual images.

Satisfies FR-106 (Attachments).

The request_attachments table refers to the service_requests table (many-to-one, via request_id) and the users table (many-to-one, via uploaded_by). It is not referred to by any other table.

Columns

- attachment_id: A primary key used to uniquely identify each attachment.
- request_id: A foreign key identifying the service request the file was uploaded to. Deleting the request deletes its attachment records.
- file_name: The original name of the uploaded file.
- file_path: The location where the file is stored.

- `file_size_bytes`: The size of the file, in bytes.
- `file_extension_type`: The file extension or type, such as pdf or png.
- `uploaded_by`: A foreign key identifying the user who uploaded the file, in the users table.
- `uploaded_at`: When the file was uploaded.

Indexes and constraints

- Primary key on `attachment_id`.
- Index on `request_id`, used to find all attachments for a request.
- Index on `uploaded_by`, used for user-specific upload tracking and storage quota monitoring.

6.1.8 The request_comments Table

The `request_comments` table records notes, status updates, resolution details and communications for a request, providing an audit trail of actions and decisions.

Satisfies FR-204 (Resolution Information) and FR-302 (Auditing).

The `request_comments` table refers to the `service_requests` table (many-to-one, via `request_id`) and the `users` table (many-to-one, via `commented_by`). It is not referred to by any other table.

Columns

- `comment_id`: A primary key used to uniquely identify each comment.
- `request_id`: A foreign key identifying the service request the comment belongs to. Deleting the request deletes its comments.
- `commented_by`: A foreign key identifying the user who wrote the comment, in the users table.
- `comment_type`: The kind of comment. Can be one of: ACTION, RESOLUTION, NOTE, UPDATE. Defaults to NOTE.
- `content`: The text of the comment.
- `created_at`: When the comment was written.
- `updated_at`: When the comment was most recently edited.
- `is_deleted`: Indicates whether the comment has been deleted. Comments are flagged rather than removed, so that the audit trail is preserved. Defaults to FALSE.

Indexes and constraints

- Primary key on `comment_id`.
- Index on `request_id`, used to retrieve the comments of a request.
- Index on `commented_by`, used to filter comments by user.
- Index on `created_at`, used to retrieve comments in chronological order.

6.1.9 The request_categories Table

The `request_categories` table defines the types of service request and routes each type to the appropriate department. It also holds the estimated time target for resolving a request of that type, and categorises requests for reporting and analysis.

Satisfies FR-101 (Categorization), FR-207 (Department-Relevant Access) and FR-304 (Request Analysis).

The `request_categories` table refers to the `departments` table (many-to-one, via `department_id`) and is referred to by the `service_requests` table (one-to-many, via `category_id`).

Columns

- `category_id`: A primary key used to uniquely identify each category.
- `name`: The name of the category. Must be unique.
- `description`: A description of the category.

- `department_id`: A foreign key identifying the department responsible for requests in this category, in the `departments` table. A department can manage one or more categories.
- `target_resolution_time`: The estimated time within which a request of this category should be resolved.
- `is_active`: Indicates whether the category is available for new requests. Setting this to `FALSE` serves as an alternative to deleting a category.
- `display_order`: The position in which the category is listed. Defaults to 0.
- `created_at`: When the row was created.

Indexes and constraints

- Primary key on `category_id`.
- Unique on `name`.
- No secondary indexes are defined in the current script.

6.1.10 The `request_status_history` Table

The `request_status_history` table maintains a complete audit trail of request status changes, recording when each transition occurred and who made it.

Satisfies FR-302 (Auditing).

The `request_status_history` table refers to the `service_requests` table (many-to-one, via `request_id`) and the `users` table (many-to-one, via `changed_by`). It is not referred to by any other table.

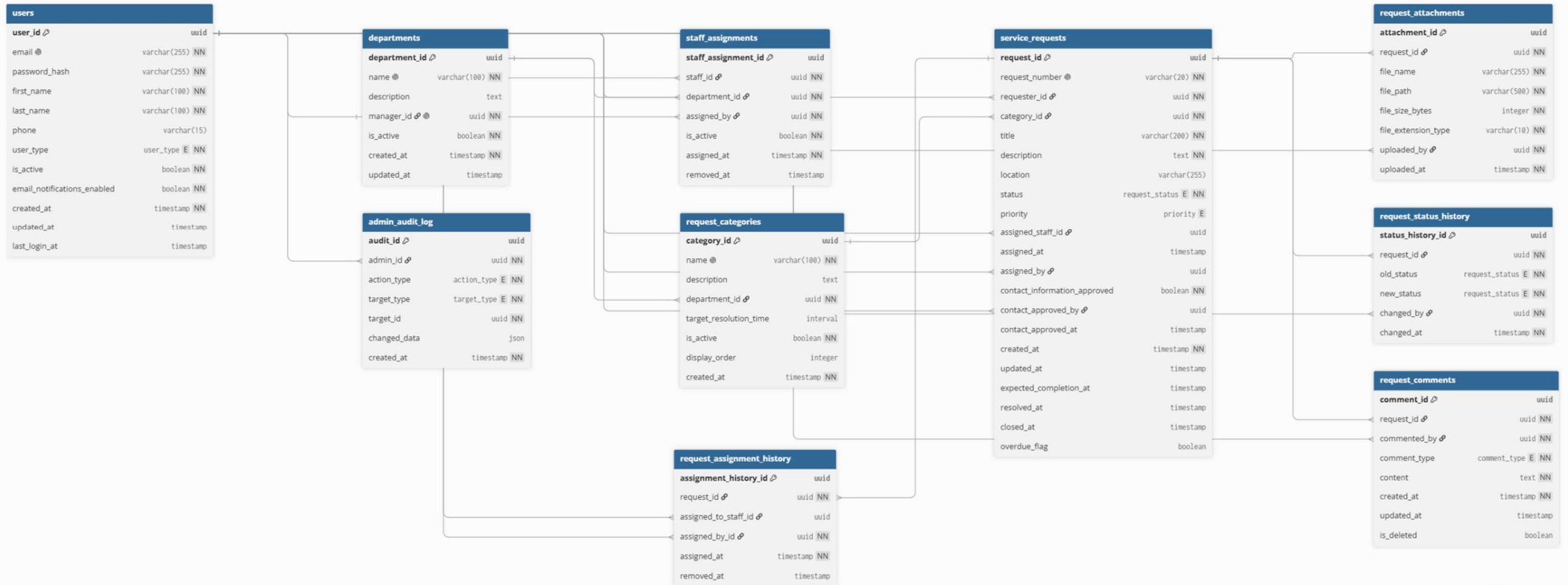
Columns

- `status_history_id`: A primary key used to uniquely identify each status change.
- `request_id`: A foreign key identifying the service request whose status changed. Deleting the request deletes its status history.
- `old_status`: The status before the change. Can be one of: `SUBMITTED`, `ASSIGNED`, `IN_PROGRESS`, `RESOLVED`, `CLOSED`, `REJECTED`.
- `new_status`: The status after the change. Can be one of the same values as `old_status`.
- `changed_by`: A foreign key identifying the user who changed the status, in the `users` table.
- `changed_at`: When the status was changed.

Indexes and constraints

- Primary key on `status_history_id`.
- Index on `request_id`, used to retrieve the status progression of a request.
- Index on `changed_at`, used for timeline queries.

6.2 Data Model / ERD / Schema



The diagram is also viewable online at dbdiagram.io here: https://dbdiagram.io/d/CivicConnect_BugBusters-6ab7b2250f25a52d0113c01b

6.3 Data Ownership and Lifecycle

Entity	Created / owned by	Read / updated by	Lifecycle / retention rule
users	Requester self-registration. Staff registered via manger / admin. Manger profile created via admin user.	Own profile; Management/Admin for authorised administration.	Accounts are not hard-deleted by default. “is_active” is set to false when access is removed. This preserves request, assignment and audit history.
admin_audit_log	System-generated for administrative actions.	Admin/Management for audit and accountability.	Append-only. Stores action_type, target_type, target_id, changed_data and timestamp. Used for administrative user, department, category and assignment changes.
departments	Management/Admin.	Management, Staff, Admin.	Departments are deactivated with is_active = false rather than deleted where they are linked to requests, categories or staff assignments. manager_id enforces one manager per department.
staff_assignments	Management/Admin. Staff can also auto assign but managers can reassign after that, manager assignment is final.	Management, Admin, relevant Staff.	Tracks which department a staff member belongs to. Active assignment is controlled by is_active and removed_at. Historical membership remains for accountability.
service_requests	Requester creates the request.	Requester sees own request; assigned Staff and Management see permitted details according to role/department.	Central operational record. Status lifecycle is SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED and lastly CLOSED, with REJECTED as a exception state. Requests should not be hard-deleted in normal operation because they support tracking, reporting and audit.
request_assignment_history	System-generated on controlled status transition.	Management/Staff for accountability.	Append-only. Records who assigned whom and when. Supports FR-303 and FR-302.
request_attachments	Requester uploads; system stores metadata and file path.	Requester, assigned Staff/Management where authorised.	Attached images belong to the parent request. They are retained with the request.
request_comments	Staff add comments; Important to note requesters cannot add comments.	Requester may see permitted feedback; Staff/Management see relevant comments.	Comments are retained for audit and resolution evidence. “is_deleted” supports soft deletion rather than physical removal.
request_categories	Management/Admin.	Requesters use categories when submitting; Staff/Management use them for routing and reporting.	Categories may be deactivated with “is_active = false”. Existing requests retain their category reference.
request_status_history	System-generated for administrative actions.	Staff/Management for audit and lifecycle visibility.	Append-only and immutable. Every status change must create a history row. It must not be silently overwritten.

6.4 Integrity, Consistency and Concurrency

CivicConnect must preserve the correctness of request, assignment, status, comment and audit data even when multiple users interact with the same request at the same time. The integrity and concurrency approach therefore focuses on database constraints, application-managed transactions, validation at the correct layer, and optimistic concurrency control for the request workflow.

Integrity constraints

The following integrity rules are enforced across the data model:

- **Entity integrity:** Every table has a primary key. Core entities such as users, service_requests, request_categories, departments and staff_assignments cannot contain NULL primary keys.
- **Referential integrity:** Foreign keys enforce relationships such as service_requests.requester_id relates to users.user_id, service_requests.category_id relates to request_categories.category_id etc.
- **Domain integrity:** Controlled values are enforced with CHECK constraints or equivalent application validation. For example, service_requests.status must be one of SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED or REJECTED.
- **Business integrity:** A request cannot move directly from SUBMITTED to CLOSED without passing through the permitted workflow states. A staff member may only update a request assigned to them and within their department. Requester contact information is only released when contact_information_approved = true and contact_approved_by is set.
- **Uniqueness:** users.email is unique. A partial unique index on staff_assignments(staff_id) WHERE is_active = true enforces that a staff member has only one active department assignment at a time, while still preserving historical assignment records.
- **Soft deletion over hard deletion:** Users, departments and requests are deactivated rather than physically deleted where they are linked to audit, assignment or request history. This preserves accountability and referential integrity.

Transactional consistency

The most important consistency boundary is the service request status transition. When an authorised staff member changes a request from one status to another, the following operations must be treated as one atomic transaction:

1. Validate the requested transition against the workflow rules.
2. Update service_requests.status, updated_at.
3. Insert a record into request_status_history.
4. Insert a resolution or progress comment into request_comments where required.
5. Insert an assignment history record where assignment or reassignment occurs.
6. Commit the transaction.

If any step fails, the entire transaction is rolled back. This prevents partial states such as a request being marked RESOLVED without a resolution comment or status history record.

Email notification is deliberately outside this transaction. The status change is the primary business operation; notification is a secondary concern. If SMTP delivery fails, the status transition is not rolled back. The failure is logged, and the notification may be retried or marked as failed for audit purposes.

Concurrency control

Concurrency control is the set of mechanisms used to manage simultaneous access to shared data so that multiple users or processes do not interfere with each other and leave the data in an inconsistent state (Databricks, 2026). It is a core part of transaction management and database isolation (Databricks, 2026). Without it, operations that happen at the same time can produce incorrect results, such as lost updates, dirty reads, or inconsistent request states (Databricks, 2026).

Concurrency control matters for CivicConnect because multiple stakeholders can act on the same service request at the same time. Staff can update status, Management can reassign or approve contact information, and Admins can manage users and departments. If two users change the same request without coordination, one change can overwrite the other.

For example:

- A manager reassigns a request to a different staff member while the original staff member is updating the status. The assignment and status could become inconsistent.

There are two main approaches to concurrency control. Pessimistic concurrency control locks data before it is changed, for example using `SELECT FOR UPDATE`, so other transactions must wait (Databricks, 2026). This is safe but can reduce performance and cause waiting or deadlocks (Databricks, 2026). Optimistic concurrency control does not lock rows in advance (Databricks, 2026). Instead, it checks whether the data changed since it was read, typically using a version column. An update only succeeds if the version still matches. If zero rows are affected, the system detects a conflict and can ask the user to refresh or retry. Databases also provide isolation levels such as Read Committed, Repeatable Read, and Serializable, which define how strictly concurrent transactions are separated (Databricks, 2026).

We have not yet determined which concurrency control approach will be used for CivicConnect. Optimistic concurrency control is a candidate because it avoids long locks and fits the low-to-medium volume expected for the platform. Pessimistic locking may be considered for critical sections where conflicts are likely. The decision will be weighed in Milestone 3 after evaluating trade-offs between correctness, performance, complexity, free-tier database capabilities, and team capability. Until then, the baseline assumes that concurrency control is required. The final choice and implementation evidence will be recorded in M3.

6.5 Scalability, Availability, SPOF & Backup/Recovery

CivicConnect is a low-to-medium volume community service request platform built by a three-person team on free-tier or low-cost infrastructure. The operational baseline therefore prioritises simplicity, recoverability and acceptable availability over high-availability enterprise architecture.

Scalability

The selected Node.js/Express application is stateless at the request-handling level. This means it can be restarted or redeployed without losing application state. For the expected initial workload, a single free-tier application instance is sufficient.

Scaling considerations:

- **Database connections:** Node.js is single-threaded, and opening a new PostgreSQL connection per request would exhaust free-tier connection limits. CivicConnect will use a connection pool through node-postgres and the managed provider's connection pooler.

- **Database scaling:** PostgreSQL provides vertical scaling and indexing. The existing indexes on `service_requests` support dashboard filtering, status queries, assignment lookup and requester history. Further optimisation, such as materialised views, is deferred until real usage data exists.
- **Free-tier limits:** Storage, bandwidth, database size and email sending limits must be monitored. Image upload size limits and attachment storage are open decisions that directly affect scalability and cost.

Availability

CivicConnect targets **best-effort availability** on free-tier infrastructure. No formal uptime SLA is claimed in M2.

Availability measures:

- The Express application exposes a `/health` endpoint used by CI and later by monitoring.
- The hosting platform can automatically restart failed application instances.
- Deployment is controlled through GitHub Actions and the staging branch, reducing the risk of broken releases reaching production.
- Email notification failure does not block request submission, status updates or resolution. This provides graceful degradation of a secondary feature.

Potential bottlenecks identified

The database bottleneck analysis identifies the points in the persistence layer where CivicConnect is most likely to experience performance decline, resource exhaustion, or unexpected cost as usage grows.

1. Dashboard and Search Query Cost

This bottleneck arises when Management dashboards, request lists, and search or filter operations scan large tables or trigger $N + 1$ loading of comments and attachments. It is important because slow dashboards directly affect Staff and Management productivity. Mitigation is to rely on the existing indexes on `service_requests`, prefer joins or batched queries over $N + 1$ access, and defer materialised views and read replicas until real usage data justifies them.

2. Growth of Append-Only Audit and History Tables

This bottleneck concerns tables such as `request_status_history`, `request_assignment_history`, `request_comments`, and `admin_audit_log`, which only grow and never shrink. It is important because continuous growth can slow history queries over time and consume free tier storage, threatening both performance and cost constraints. Mitigation is that normal indexing is sufficient at current volume, while partitioning, archiving, or retention rules for non critical audit data are deferred until usage evidence shows they are needed.

3. Free-Tier Resource limits

This bottleneck refers to the limits on database size, connections, bandwidth, storage, and backup retention imposed by free tier providers. It is important because the system may work in development but hit a paywall or degrade in production, which would violate the low cost constraint and risk unexpected redesign. Mitigation is to monitor database size, connection usage, and storage, keep large binaries out of PostgreSQL, store images as metadata plus external file references, finalise image size limits and attachment storage before production, and keep backups within provider limits while testing restore procedures.

4. Attachment Metadata Coupling

This bottleneck involves `request_attachments` storing metadata and file paths, where storing files

locally on the application instance makes them non-durable and lost on redeploy, while external storage adds another dependency. It is important because lost attachments undermine the evidence needed for request resolution and auditability. Mitigation is to store only metadata and a stable path or reference in PostgreSQL, store actual images in durable external object storage, and confirm the final storage decision in Milestone 3.

Backup and Recovery

Recovery Time objective:

The target we are going to aim for is 8 hours.

The system is important for operational continuity but is not life-critical. An 8-hour restoration window is acceptable for the initial baseline.

Recovery Point Objective:

The target we are going to aim for is 24 hours.

Daily backups mean up to one day of request data could be lost in a catastrophic failure. This is acceptable for the initial low-volume context.

Backup approach:

- CivicConnect will rely on the selected managed PostgreSQL provider's automated daily backups.
- Backups are stored separately from the application instance.
- Database backups cover all relational data: users, departments, staff assignments, service requests, request categories, status history, assignment history, comments, attachments metadata and admin audit logs.
- Attachment backup depends on the final image storage decision. If images are stored in external object storage, that provider's retention and backup settings must be configured.

Point-in-time recovery:

Still to be determined. Based on database hosting and deployment, it will be considered if we will make use of point in time recovery, for now we defer this decision.

Recovery procedure:

1. Detect database failure or data corruption.
2. Notify the team and pause production writes if possible.
3. Restore the latest automated backup to a new database instance.
4. Update the application connection string through environment variables or secrets.
5. Run smoke tests against /health, login, request submission and status transition.
6. Resume service and record the incident in the audit or engineering log.

Testing requirement:

A restore test must be performed at least once before production release. A backup that has never been restored is not considered verified evidence. This aligns with the assignment 3 production-readiness concern that backup and recovery procedures must be tested, not merely documented.

7. Technology Stack Baseline

7.1 Technology Requirements and Selection Criteria

CivicConnet's technology stack was not evaluated on industry trends but on survivability criteria for a 3-person team operating on a short timeline. The main selection drivers include:

- Cognitive load (human focus) and context switching. The architecture must minimize the number of disparate languages, frameworks, and build pipelines the team must maintain.
- Operational cost (NFR-301). The stack must have a memory and compute usage small enough to avoid penalties and out-of-memory crashes on standard PaaS free tiers which is usually capped at 512MB to 1GB RAM per instance or service.
- Concurrency, the persistence layer must support transaction isolation and Optimistic Concurrency Control (OCC) to mitigate, "lost update" anomalies during a case where simultaneous ticket assignments/updates exist.
- Ecosystem maturity, tools should offer strong out-of-the-box CI/CD support without requiring shell scripting (the practice of writing shell scripts, which are text files containing a sequence of commands designed to be executed by a Unix shell a command-line interpreter).

7.2 Alternatives considered

- Decoupled SPA (React/ Next.js) + API Backend: Rejected, while this is standard for enterprise systems, an SPA introduces distributed state management, configuration overhead and require maintaining two separate CI/CD build pipelines. Given our schedule, this doubles deployment complexity and is not needed for what is essentially a forms and dashboard application.
- Java Servlets / Spring Boot: Rejected, the JVM's baseline memory and garbage collection overhead makes it a problem to free-tier cloud environments. Spring Boots heavy abstraction layers would also slow down initial scaffolding compared to a lighter runtime environment, thus making our milestone schedule tighter.

7.3 Selected Technology Stack

The team has baselined a JavaScript-centric MVC Web-Application to maximize developer power and minimize the integration boundaries.

- Runtime and API framework: Node.js (v24) with Express.js. Node's asynchronous, event driven I/O model is suited for CivicConnect, which is heavily I/O bound, database queries and SMTP calls, rather than CPU processing bound.
- Frontend Presentation: We chose server-side rendering (SSR) via EJS (Embedded JavaScript) coupled with Tailwind CSS. This removes client-side state synchronization issues and delivers fast initial page loads. Tailwind also eases development by having the advantage of allowing us to adapt the application to different screen sizes, which satisfies NFR-401 (Compatibility).
- Database and persistence: PostgreSQL. This is chosen for its strict relational integrity, it's support for row-level locking (concurrency control mechanism that allows multiple transactions to access and modify different rows within the same table) and advanced JSONB indexing which provides unstructured audit payloads are required later.
- Database access layer: Node-Postgres, using transaction blocks to guarantee atomicity during request state transitions.
- Integration: We chose Nodemailer for SMTP, which allows environment switching between Ethereum for testing and then Gmail for production.

- Quality Gates: We chose Jest for unit/integration testing and ESLint for static analysis, this is directly integrated into GitHub Actions.

7.4 Versions and Compatibility Assumptions

- Runtime parity: Node.js v24 is pinned via .nvmrc (The .nvmrc file is a configuration file used by Node Version Manager (NVM) to specify the required Node.js version for a project. It is a simple text file placed in the project's root directory containing a version string, such as v18.16.1) The GitHub Actions CI pipeline is configured to read this file to prevent the, “works on my machine” deployment anomalies.
- Database connection pooling. Because Node.js is single-threaded, opening a new TCP connection to PostgreSQL per request will drain free-tier database connection limits. We assume the selected hosting provider Supabase/Neon will provide a connection pooler (Supabase=Supavisa; Neon=PgBouncer both built-in) at the infrastructure level.

7.5 Technology Trade-offs

Advantage: Unifying the stack under JavaScript (Node/Express/EJS) allows all team members to comfortably debug across the request lifecycle. Server-side rendering reduces the areas for security vulnerabilities because the server controls the DOM generation.

Trade-off: EJS relies on full-page reloads, which means a less dynamic UX compared to React/Next.js. We are trading client-side polish for simplicity, thereby prioritizing backend stability and security.

7.6 Evidence and traceability

- ADR reference: ADR-004 (Technology Stack Selection – MVC Node/PostgreSQL).
- Implementation evidence: .nvmrc, package.json, ESLint configuration and the express application passing the Jest CI health check on the testing branch.

8. Initial Design Decisions

8.1 Design decision 1: Decoupling domain logic from external side-effects

Problem / Context:

When a staff member resolves a ticket (FR202 and FR206) the system performs a primary domain operation (updates database). However, it must also trigger side-effects: writing to the audit log (FR-302) and dispatching an email (FR-104). Then tying a network operation (SMTP) directly to the database transaction creates performance risks; if the email server times out, the database transactions remain open thereby holding locks and degrades the system throughput.

Alternatives considered:

Assignment 2 evaluated Direct Service Orchestration synchronous, procedural execution against an Observer / Event-Based Approach asynchronous decoupling.

Selected Pattern: Observer / Event-Based Approach

The RequestWorkflowService executes the database transaction. When there is a successful commit, then it fires an in-memory event for example, RequestStatusChanged. The NotificationService and AuditLogService act as decoupled listeners that react to this event asynchronously in the background.

Why it fits CivicConnect:

“Eventual consistency” is enforced for secondary actions and aligns with the API integration decision which means that SMTP failure must not roll back the main database state. The user gets an HTTP 200 OK response as soon as the database updates while the slow SMTP dispatch happens safely outside the request thread.

Affected components:

- RequestWorkflowService, event publisher.
- NotificationService, subscriber/ SMTP Handler
- AuditLogService, subscriber

Trade-off / Complexity:

In-memory event emitters do not survive crashes. If the Node.js process dies then milliseconds after the database commits but before the email is sent, then the email is lost. Implementing a persistent message queue like RabbitMQ would solve this but it violates our constraint to avoid operational complexity. We accept this risk as it is appropriate for a not so critical community portal.

Implementation evidence and traceability:

- ADR Reference: ADR-005 (Event-driven side-effect management).
- Evidence: The architecture boundary for this is established via the src/services/emailService.js stub (documented in section 13.2). As per our M2 milestone limitations the final Node.js EventEmitter logic is deliberately deferred and tracked as “Pending - M3” in the RTM for FR-104.

8.2 Design Decision 2: Centralized state mutation Governance

Problem / Context:

The service request lifecycle: SUBMITTED > ASSIGNED > IN_PROGRESS > RESOLVED > CLOSED, involves authorization rules. By placing if / else statements inside individual UI controllers creates logic duplication, thus making it likely that a security vulnerability exists when modifying state transitions in the future.

Alternatives considered:

Assignment 2 compared the State Pattern against a procedural Centralised Workflow / Transition Service.

Selected Pattern: Centralised Workflow / Transition Service

We implement a finite state machine configuration within a singular RequestWorkFlowService. All request mutation attempts are routed through this boundary layer, which validates the payload against an allowed transitions map before generating the SQL update query.

Why it fits CivicConnect:

The State Pattern requires creating classes for every single state for example, OpenState, ResolvedState. This brings unneeded object allocation overhead and boilerplate for a static 5 step lifecycle. A centralised service provides the same security and consistency through simple guard clauses, ultimately optimizing for maintainability.

Affected components:

- RequestWorkflowServices, boundary layer and rules engine.
- RequestController, responsible for HTTP parsing and delegation.

Trade-off / Complexity:

The service class can become dense. We will mitigate this by extracting the validation rules into utility functions thus ensuring the central service acts only as an orchestrator and not a monolithic script.

Implementation evidence and traceability:

- ADR Reference: ADR-006 (Centralised Workflow FSM).
- Evidence: The separation of concerns is structurally prepared in the repository via the `src/services` and the `src/controllers/` directories.

9. Initial API / Integration Decisions

9.1 Interface/Integration background and context

For CivicConnect we added the requirement that requesters have to option to receive feedback when a request status changes (FR-104; in-scope feature: "Feedback via email push notification" from PED v1). The Request Management module (FR-202, FR-206) is the producer of status-change, while an email notification component is the consumer responsible for delivering the message to the requester.

During our Assignment 2 research, three integration approaches were identified and evaluated: HTTP/REST API, asynchronous event/message (broker-based), and SMTP. The asynchronous broker approach was rejected due to the operational complexity for a non-critical messaging system (it is intended for applications where message delivery must be ensured, such as financial transactions or healthcare communications). The final choice between REST-based API and SMTP was deferred to Milestone 2 pending cost and scalability analysis.

For M2 we have now reached a meaningful implementation decision. The decision has been finalised: SMTP via Nodemailer is the chosen choice. As it is a Node.js library that handles SMTP connections directly (fits our tech stack). This is also a stable library that is widely used (Patel, 2026). Further justification follows below:

9.2 Responsibilities and Information Exchange

The Request Management component is responsible for detecting a controlled status transition (e.g., "In Progress" to "Resolved") and triggering the notification. It does not need to know how the email is ultimately delivered.

The Notification component (implemented as a Nodemailer service module) is responsible for:

- Resolving the requester's email address from the Requester ID.
- Composing the email message from the status-change data.
- Connecting to the configured SMTP server and dispatching the message.
- Logging the outcome (success or failure) for auditing and troubleshooting.

The information exchanged is derived from assignment 2 includes:

- Requester ID (used to look up the email address).
- Previous status and new status.
- Timestamp of the status change.
- Staff member ID (for name and department lookup).
- Optional comment added by the staff member (FR-204).

Structure of the email notification :

‘Good day [requester name],

This email is with regards to [request name] with request id of [request id]. The status of the request has changed from status [prev status] to [new status]. Staff member ` [staff member name] ` had this to say:

[comment by the staff member]

Kind regards,

CivicConnect, [department name] department’

9.3 Interaction Mechanism

The selected mechanism is direct SMTP integration using Nodemailer. This approach was chosen over a managed REST-based email API for the following project-specific reasons:

Factor	SMTP + Nodemailer	REST API (e.g SendGrid)
Vendor lock in	Minimal, only the SMTP credentials need to change if the provider is switched.	Higher, application depends on the provider's API contract and version
Cost	Nodemailer is free and open-source; the only cost is the SMTP service (many free/low-cost options exist, e.g., Gmail for low volume).	Free tiers exist but scale into per-email pricing; cost increases with volume
Operational Complexity	Moderate, Nodemailer handles connection, authentication, and message dispatch.	Moderate, requires managing API keys, rate limits, and provider-specific error codes.
Future sustainability	High, switching SMTP providers requires only configuration changes, not code changes.	Lower, switching providers may require code changes to accommodate different API contracts.

Testing and production configuration:

- **Development/Testing:** Ethereum Email, a free fake SMTP service designed for Nodemailer. Nodemailer can automatically generate test credentials via `nodemailer.createTestAccount()`, and sent messages are captured in a web interface without being delivered to real recipients. The preview URL from `nodemailer.getTestMessageUrl(info)` allows us as developers to inspect the message format and content.
- **Production:** Gmail SMTP (via service: "gmail" preset) is the initial production target for low-volume sending. An App Password (16-character, generated with 2-Step Verification enabled) is used for authentication. The service: "gmail" shortcut automatically configures the SMTP host and port settings. A centralised transport factory will switch between Ethereum and Gmail based on the `NODE_ENV` environment variable, so the application code remains identical across environments.

9.4 Validation, Errors and Security Boundary

This section outlines the planned approach for validation, error handling, and security for the SMTP-based email notification integration. Implementation of these controls is scheduled for Milestone 3.

Validation

The notification service will validate that the requester's email address has been successfully verified before attempting to send. This prevents unnecessary SMTP connection attempts when the recipient address is missing or malformed. Address validation is a recognised security control for email libraries, as user-supplied addresses can be an injection surface if not properly handled (Patel, 2026).

Error/Failure Behaviour

If the SMTP connection fails or the server returns an error, the notification service will log the failure and mark the notification as failed in the audit trail. Notification failure will not roll back the status transition.

For transient SMTP failures (e.g., temporary network issues), a single retry may be attempted. Nodemailer's direct transport module supports automatic retry with a configurable delay for messages that cannot be sent immediately, handling errors and greylisting (Nodemailer, 2025). Duplicate notifications will be avoided by tracking a notification ID for each status-change event. This addresses the concern about "spamming the user with emails unnecessarily".

Junk mail placement is a failure mode even when SMTP reports successful delivery. To reduce this, we plan to possibly implement SPF, DKIM, and DMARC authentication, keep email content simple and factual, and encourage requesters to whitelist the sender address (This decision is not finalized, and will still be reviewed in M3).

Security Boundary

SMTP credentials (username and App Password) will never be committed to the repository. They will be injected via environment variables or CI secrets. This is the recommended practice for Nodemailer configurations, where sensitive options such as passwords can be overwritten using environment variables (Nodemailer, 2025)

Connections to Gmail SMTP will use the recommended secure ports and TLS/STARTTLS settings as per Nodemailer's official documentation, to protect credentials and message content in transit.

Email content must respect the requester's and staff member's privacy (NFR-101, FR-305). The message template will include only the information necessary for the notification and will not expose internal identifiers or sensitive data beyond what the requester is entitled to see.

Change/Version Implications

Nodemailer's SMTP transport documentation notes that with SMTP, switching providers is a matter of changing the configuration object or connection URL, whereas API-based sending creates vendor lock-in (Nodemailer, 2025).

10. Deployment Compatibility

10.1 Deployment Direction and Environment Assumptions

CivicConnect is being engineered as a low-to-medium volume, cloud-hosted web application for a community-focused organisation. The deployment approach prioritises simplicity, recoverability, acceptable availability and low operational cost rather than enterprise-scale redundancy or distributed infrastructure. This is consistent with the current PED, which assumes that a single application instance is sufficient for the expected initial workload.

The current direction is to use a managed Platform-as-a-Service (PaaS) environment for the eventual production deployment. Railway and Render remain deployment candidates, but the final provider has not yet been selected. The choice will depend on factors such as runtime support, PostgreSQL compatibility, resource limits, configuration and secrets support, operational simplicity and cost.

Project Conditions

Contextual factor	Current assumption
Scale	CivicConnect is expected to operate at a low-to-medium workload. One application instance is considered sufficient initially.

Team	The system is being developed by a three-person team, so the deployment infrastructure must remain practical to configure, maintain and troubleshoot.
Schedule	Deployment decisions must support the limited milestone schedule without introducing unnecessary infrastructure complexity.
Cost	Free or sufficiently low-cost hosting and supporting services are preferred for the project.
Availability	Best-effort availability is acceptable at this stage. No enterprise-level uptime SLA is claimed.
Scalability	Future scaling should remain possible, but additional infrastructure will only be introduced when supported by actual evidence or usage requirements.

These assumptions align with ASR-05: Cost-Effective Deployability and Resource Efficiency, which requires CivicConnect to remain executable in the team's development environment and deployable using a free or sufficiently low-cost environment appropriate to the project. The ASR also requires hosting, database, storage and supporting-service limitations to be identified before final commitment and requires the deployment approach to remain practical for the three-person team to configure, test and maintain.

Runtime Environment

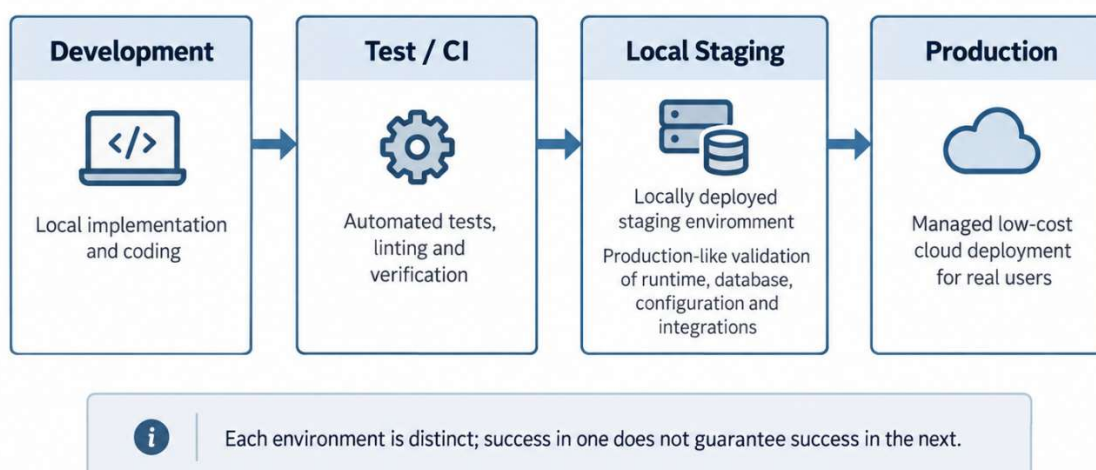
The runtime environment follows the technology baseline already established in the PED:

- Node.js v24 with Express.js for the application runtime.
- EJS and Tailwind CSS for server-side presentation.
- PostgreSQL for persistent data, accessed through node-postgres.
- Nodemailer/SMTP for email notifications.
- GitHub Actions, Jest and ESLint for CI and verification rather than production runtime functionality.

The application is assumed to remain stateless at the request-handling level, while persistent business data is stored outside the application process. This allows the application to be restarted or redeployed without losing persistent system data.

Environment Progression

CivicConnect will distinguish between the following environments:



Before moving towards production deployment, CivicConnect will first be deployed locally in a dedicated staging environment. This staging environment will be used to run the application as an integrated system under conditions that are as close as reasonably possible to the intended production configuration. This will allow the team to verify areas such as runtime compatibility, database

connectivity, environment-specific configuration and supporting integrations before committing to a final cloud deployment environment.

Using a local staging environment also allows the team to identify deployment-related problems without prematurely committing to a production hosting provider. This follows the Master Project Brief requirement to distinguish between development, test, staging and production environments because successful operation in one environment does not automatically prove that the system will operate correctly in another.

The exact production host, managed database provider, attachment-storage solution and automated deployment process remain deliberately open until sufficient evidence is available. The current PED already records the Render-versus-Railway hosting decision and automated deployment as deferred decisions.

10.2 Compatibility of Architecture, Data and Technology Decisions

The deployment direction established in Section 10.1 is compatible with the architecture, persistence and technology decisions already baselined for CivicConnect. The selected solution can initially operate as a single application instance in the local staging environment and later move to a managed cloud production environment without requiring a fundamental redesign. The table below shows how the main engineering decisions already made in the PED support this deployment direction.

Engineering Area	Current Decision	Deployment Compatibility
Architecture	Modular Monolith	CivicConnect remains a single deployable application while responsibilities are separated into internal modules. This fits the single-instance deployment direction and avoids the extra networking, coordination and operational complexity associated with distributed services.
Application Runtime	Node.js v24 with Express.js	The application can run as a single web process in both the local staging environment and later PaaS production deployment. Pinning Node.js v24 through .nvmrc also supports runtime consistency across environments.
Presentation	EJS with Tailwind CSS	EJS is rendered by the same Node.js application, so no separately deployed frontend application is required. Tailwind CSS remains part of the application presentation layer and introduces no additional runtime service.
Persistence	PostgreSQL	Persistent business data is kept outside the application process and can later be hosted through a managed PostgreSQL service. This allows the application to be restarted or redeployed without losing persistent data.
Database Access	node-postgres with connection pooling	node-postgres fits the selected Node.js runtime and supports transaction handling and controlled PostgreSQL access. Connection pooling also helps reduce the risk of exhausting managed-provider connection limits.
Application State	Stateless request handling	Persistent business state is not tied to the running application instance, which supports restart, redeployment and possible future horizontal scaling.
Email Integration	Nodemailer using SMTP	Email is treated as an external supporting integration rather than a separate CivicConnect deployment unit. SMTP configuration can therefore vary between environments without changing the core application architecture.
Verification / CI	GitHub Actions, Jest and ESLint	Automated verification can occur before changes progress into staging. These tools support the deployment process without adding runtime infrastructure to the deployed application.

The selected Modular Monolith, Node.js/Express runtime, PostgreSQL persistence and supporting integrations are therefore plausible for the deployment direction defined in Section 10.1. The current PED already establishes CivicConnect as a single deployable application, with persistent data separated from the application process and deployment complexity intentionally kept proportionate to the project.

10.3 Deployment Configuration, Secrets, Persistence/State and Networking Implications

The deployment direction established in Sections 10.1 and 10.2 creates several configuration, security, persistence and networking implications that must be considered before CivicConnect progresses

from local staging to production. The purpose of this section is to identify those implications at the level currently known in M2 without prematurely finalising provider-specific production configuration.

Area	Current Implication
Runtime Configuration	Node.js v24 is pinned through .nvmrc so that developer, CI and later deployment environments use the same runtime version. Environment-specific behaviour should be controlled through configuration rather than changes to application code.
Environment Variables	Values that differ between development, staging and production must be externalised from the source code. This includes database connection details, session configuration, SMTP configuration and other provider-specific settings.
Secrets	Sensitive values must not be committed to the GitHub repository. The current CI process already injects SESSION_SECRET using GitHub Actions repository secrets. Production secrets will later be supplied using the selected deployment platform's secure environment-variable or secret-management mechanism.
Database Persistence	PostgreSQL remains the persistent store for CivicConnect business data. Persistent data must remain independent of the running Node.js application instance so that restarting or redeploying the application does not remove system records.
Application State	Request handling is intended to remain stateless at application-instance level. If CivicConnect later runs across more than one application instance, session state will need to be externalised so that users are not tied to a particular instance.
Attachment Persistence	Images must not rely on temporary application-instance storage for production because locally stored files may be lost when an application instance is restarted or redeployed. The current PED therefore anticipates durable storage for attachments, while the final storage mechanism remains deferred.
Database Connectivity	The Node.js application requires network connectivity to PostgreSQL. node-postgres connection pooling will be used, together with provider-side pooling where available, to avoid exhausting managed database connection limits.
Client Connectivity	Users will access CivicConnect through the web application. Production access will require a secure public endpoint, while the local staging deployment remains locally hosted for controlled validation.
SMTP Connectivity	The application requires outbound network access to the configured SMTP service. Development/testing can use Ethereum, while Gmail SMTP is currently the initial production direction. SMTP credentials must be supplied through environment configuration rather than source code.
External Service Failure	Network-dependent supporting services must not unnecessarily control core CivicConnect transactions. In particular, an SMTP failure must not roll back a successfully completed request-status change.
Health and Failure Detection	The Express application already provides a /health endpoint, which can be used during staging and later deployment to confirm that the application starts and responds successfully.

These implications establish the deployment requirements currently known in M2. Node.js runtime consistency, externalised configuration, controlled secrets, persistent PostgreSQL storage, stateless request handling, connection pooling and failure isolation are already aligned with the current PED baseline. The exact production values, provider-specific secret management, final attachment-storage mechanism and any additional networking configuration will be finalised only once the relevant deployment providers and services are selected.

10.4 Deployment Compatibility, Cost, Availability and Operational Risks

The deployment products and supporting services currently selected or being considered for CivicConnect were reviewed against the deployment direction established in Section 10.1. The assessment records each product's intended use, compatibility with the existing architecture and technology stack, expected cost for the intended project usage, practical availability limitations and material operational risks. Where a final provider has not yet been selected, current candidates are assessed separately rather than treated as final decisions. EJS and Tailwind CSS are packaged within the application and do not introduce a separate hosted service or recurring deployment cost, so they are not listed separately below.

Product / Service	Intended Use	Compatibility	Cost for Intended Use	Availability / Service Limitations	Material Operational Risk	Related PED Risk
Node.js v24 + Express.js	Runtime and web framework for the single CivicConnect application process.	Compatible with the selected Modular Monolith and PaaS deployment direction. Node.js is suitable for HTTP/database/SMTP I/O, while Express runs in the same deployable application boundary.	US\$0 software licence cost. Node.js is free and open source; Express is an open-source Node.js framework (Node.js, 2026; Express.js, 2026).	Availability depends on the hosting platform rather than Node.js/Express themselves. Runtime parity is controlled through the pinned Node.js version.	Environment/runtime mismatch or unsupported provider runtime could cause deployment failure even when the application works locally.	RSK-005; proposed RSK-011
Render - Free Web Service	Candidate cloud host for the Node.js/Express application.	Partially compatible. Supports Node web services and provides 512 MB RAM on the Free plan. However, Free web services block outbound traffic on SMTP ports 25, 465 and 587, which conflicts with the current Gmail SMTP/Nodemailer production direction (Render, 2026-a; Render, 2026-b).	US\$0/month on Free. The next 512 MB paid web-service compute plan is US\$7/month (Render, 2026-b).	750 Free instance-hours per workspace/month. Free web services spin down after 15 minutes without inbound traffic, cannot use persistent disks, and may be suspended when free instance hours are exhausted (Render, 2026-a).	Cold starts and free-tier limits may reduce responsiveness or availability. Direct Gmail SMTP will not work on Render Free. Local filesystem data is non-durable.	RSK-005; RSK-006; RSK-007; RSK-008
Railway - Free	Alternative cloud host for the Node.js/Express application.	Partially compatible. Supports public HTTP/HTTPS deployment and provides up to 1 vCPU and 0.5 GB RAM per service on Free. SMTP is disabled on Free, Trial and Hobby, so the current Gmail SMTP/Nodemailer production direction cannot operate on those tiers (Railway, 2026-a; Railway, 2026-b).	US\$0/month with US\$1 of free resource credit/month. Hobby is US\$5/month; Pro is US\$20/month. SMTP requires Pro or above (Railway, 2026-a; Railway, 2026-c).	Free has one replica, 0.5 GB RAM and 0.5 GB volume storage per service. Free-plan restart policy is limited, and workloads can be stopped if usage limits are reached (Railway, 2026-a; Railway, 2026-d).	The US\$1 monthly credit may be insufficient for continuous operation. Retaining direct SMTP would require moving to Pro, increasing cost materially.	RSK-005; RSK-006; RSK-008
PostgreSQL	Primary relational persistence technology for users, requests, workflow, history and audit data.	Fully compatible with the Node.js application through node-postgres and with the relational integrity/transaction requirements already baselined in the PED.	US\$0 software licence cost. PostgreSQL is free and open source; hosting cost depends on the selected managed provider (PostgreSQL Global Development Group, 2026).	The database software itself does not provide a hosted availability guarantee; availability, storage, backup and recovery depend on the managed provider selected.	A provider outage, capacity limit or recovery failure affects most core CivicConnect functions because PostgreSQL is the central persistent store.	RSK-006; RSK-010
Neon - Free PostgreSQL	Candidate managed PostgreSQL provider.	Compatible with standard PostgreSQL connectivity and the application's pooled connection approach. Neon's serverless model and connection options fit the low-volume deployment direction.	US\$0/month. Free includes 50 CU-hours per project/month, 0.5 GB storage per project and 5 GB egress/month (Neon, 2025).	Free compute uses scale-to-zero. Free includes Instant Restore history up to 6 hours or 1 GB of changes, whichever is smaller, and limited monitoring history (Neon, 2025).	Scale-to-zero may introduce wake-up latency. Storage/compute/egress limits may be exceeded as usage grows. Restore capability must still be tested against the PED's recovery targets.	RSK-006; RSK-010; proposed RSK-011
Supabase - Free	Alternative managed PostgreSQL provider;	Compatible with PostgreSQL and supports managed Postgres connectivity. It is	US\$0/month. Free includes 500 MB database	Free projects can be paused after 1 week of low activity. Automatic	Automatic pausing can reduce immediate availability. Lack of	RSK-006; RSK-010;

Product / Service	Intended Use	Compatibility	Cost for Intended Use	Availability / Service Limitations	Material Operational Risk	Related PED Risk
	could also provide file storage if later selected.	technically suitable, but its Free-plan availability and backup limits require attention before production use.	size per project, 5 GB egress and 1 GB file storage (Supabase, 2026a; Supabase, 2026-a).	backups are not included on the Free plan, and Free projects enter read-only mode after exceeding the 500 MB database-size quota (Supabase, 2026-a; Supabase, 2026b).	included automatic backups conflicts with the current recovery assumption unless the team implements another verified backup approach or moves to a paid plan.	proposed RSK-011
Nodemailer	Application library used by the Notification component to send email through SMTP.	Highly compatible with the Node.js stack and does not require a separate CivicConnect deployment unit. SMTP provider details can be changed through configuration.	US\$0 licence cost. Nodemailer uses the MIT-0 licence (Nodemailer, 2026).	No independent service availability because Nodemailer runs inside the application. Actual availability depends on the SMTP provider and whether the host permits outbound SMTP.	A compatible library can still fail operationally if the chosen host blocks SMTP or if the external SMTP provider is unavailable.	RSK-005; RSK-008
Ethereal Email	Development/testing SMTP service for verifying email generation without sending to real users.	Compatible with Nodemailer and appropriate for development, CI or local staging email testing.	US\$0. Ethereal describes itself as a completely free fake SMTP service for email testing (Ethereal, 2026).	Messages are captured for preview rather than delivered to real recipients; therefore it is not a production notification service.	Successful Ethereal testing does not prove that production SMTP credentials, host networking or real-world deliverability will work.	RSK-008; proposed RSK-011
Gmail SMTP	Current low-volume production SMTP direction for requester email notifications.	Compatible with Nodemailer, but only if the selected application host allows outbound SMTP. This conflicts with Render Free and Railway Free/Hobby.	No separate SMTP subscription is assumed if an existing personal Gmail account is used. If a business-domain mailbox is later required, Google Workspace Starter's standard list price is US\$7/user/month in South Africa (Google, 2026-a).	Personal Gmail accounts may hit sending limits when sending more than 500 emails in a day or more than 500 recipients in a single message (Google, 2026-b).	Sending limits, authentication problems, spam filtering or host-level SMTP blocking may stop notifications. Email failure must remain isolated from the core request transaction.	RSK-006; RSK-008
GitHub Actions	CI verification for linting, tests and controlled progression to staging.	Compatible with the existing GitHub repository, Node.js runtime and Jest/ESLint workflow. It adds no production runtime service.	For public repositories, standard GitHub-hosted runners are free. GitHub Free private repositories include 2,000 Actions minutes/month and 500 MB artifact storage (GitHub, 2026).	Availability for the project is mainly constrained by included usage, workflow health and GitHub service availability. Larger runners remain chargeable even when standard public-repository use is free.	Exhausted included usage or failed workflows can block merges/releases and delay deployment, although they do not stop an already-running application.	RSK-001; RSK-005; RSK-006

Overall Deployment Risk Position

The current products are technically compatible with the Modular Monolith, Node.js/Express and PostgreSQL architecture, but the provider research identifies two material issues that must be carried forward:

- 1. The current free cloud-hosting assumption conflicts with the selected SMTP integration. Render Free blocks outbound SMTP ports, while Railway permits SMTP only on the Pro plan (US\$20/month) (Render, 2026a; Railway, 2026b).Therefore, the team cannot currently claim that free Render/Railway + Nodemailer + Gmail SMTP forms a fully compatible production deployment. Before production, the team must either select a hosting configuration that permits SMTP, accept the required hosting cost, or reconsider an HTTPS-based transactional email provider.
- 2. The managed PostgreSQL provider must be selected against the project's actual recovery requirements. In particular, Supabase Free does not include automatic backups, which conflicts with the current PED wording that assumes automated provider backups (Supabase, 2026). Neon Free provides limited restore capability, but this still needs to be validated against the team's stated recovery objectives rather than treated as proven recovery evidence.

10.5 Deferred Deployment Decisions and Evidence Required

Several deployment decisions are deliberately not finalised during Milestone 2 because the required implementation, staging and provider evidence does not yet exist. These decisions are deferred to avoid making premature commitments that could introduce unnecessary cost, incompatibility or rework. The table below records the remaining deployment decisions and the evidence that must be produced before each decision can responsibly be finalised

Deferred Decision	Current Position	Why the Decision Is Deferred	Evidence Required Before Finalisation	Expected Finalisation
Production Application Hosting Provider	Render and Railway remain the main PaaS candidates. No final production host has been selected.	Both support the Node.js deployment direction, but Section 10.4 identified differences in free-tier resource limits, availability behaviour and SMTP support. In particular, the current direct Gmail SMTP approach is not compatible with the intended free tiers.	Deploy the integrated Node.js/Express application in a representative environment; measure memory/resource usage; verify PostgreSQL connectivity and pooling; verify environment-variable and secret handling; determine whether the selected email approach is supported; compare actual operational cost and availability behaviour.	M3, before production deployment.
Managed PostgreSQL Provider	PostgreSQL is baselined, but the final managed provider has not been selected. Neon and Supabase remain candidates.	The database technology is already decided, but provider-specific limits differ in storage, connection pooling, inactivity behaviour, backup and recovery capability.	Establish a working application database connection; test pooled connections; review actual database size and expected growth; verify provider backup/restore functionality; perform a restore test; confirm that the selected provider can support the current 8-hour RTO and 24-hour RPO expectations.	M3, before production release.
Attachment / Image Storage Mechanism	FR-106 attachments remain an open deployment decision. The application instance must not be treated as durable production storage.	Local application storage may be lost during redeployment, while storing large binary data in PostgreSQL may increase database size and performance cost. External object storage introduces an additional service and operational dependency.	Implement representative image uploads; establish expected file sizes and usage volume; measure database impact if database storage is considered; test a durable external-storage approach; verify access control, supported file types, quotas, backup/retention and cost.	M3, before attachment functionality is considered production-ready.
Production Email Delivery Approach	Nodemailer remains the application-level email integration. Ethereum is suitable for testing and Gmail SMTP is the current production direction, but the final production delivery configuration is not yet viable with the intended free hosting options.	Section 10.4 identified that Render Free blocks outbound SMTP and Railway restricts SMTP to higher plans. A final decision therefore depends on the selected host and whether direct SMTP remains practical.	Test email delivery from the selected deployment environment; confirm outbound-network restrictions; verify authentication and secret handling; assess actual delivery reliability; compare the cost of SMTP-enabled hosting against an HTTPS/API-based transactional email alternative if required.	M3, once the production host is selected.

Deferred Decision	Current Position	Why the Decision Is Deferred	Evidence Required Before Finalisation	Expected Finalisation
Production Environment Variables and Secrets Configuration	Secrets are currently externalised and GitHub Actions already uses repository secrets for CI. Final production values and provider-specific secret configuration remain deferred.	The final secret-management mechanism depends on the selected hosting, database and external-service providers.	Produce a complete list of required environment variables; configure them in the selected host; verify that no secrets are committed to GitHub; test startup with missing/invalid configuration; confirm database, session and email credentials are correctly injected.	M3, during deployment configuration.
Automated Production Deployment / CD	GitHub Actions currently provides CI and protects the staging branch. Automated deployment from staging/main to the production host is not yet implemented.	The CI pipeline is deliberately minimal in M2, and automated deployment is deferred until the hosting provider is baselined.	Finalise the production host; establish a manual deployment successfully first; verify environment variables and secrets; define release approval and rollback behaviour; then test an automated deployment pipeline with a controlled release.	M3, after the hosting provider is selected and manual deployment is proven.
Production Monitoring and Operational Observability	CivicConnect currently provides a /health endpoint, but full production monitoring and observability are not implemented.	Production observability was intentionally excluded from the M2 CI scope. Appropriate monitoring depends partly on the capabilities of the final deployment platform.	Deploy the application; verify /health; determine what logs, uptime information and failure alerts the selected host provides; define minimum monitoring and failure-detection evidence needed for release.	M3, before production readiness is approved.
Rollback and Recovery Procedure	Database recovery targets and a draft recovery process exist, but production rollback and recovery have not yet been proven.	The actual procedure depends on the selected hosting and database providers. Documentation alone is not sufficient evidence that recovery works.	Complete at least one database restore test; verify application reconnection after restoration; perform smoke tests against /health, login, request submission and status transition; establish how an unsuccessful application release will be rolled back.	M3, before production release.
Horizontal Scaling and External Session Storage	The initial deployment remains a single application instance. Horizontal scaling is not currently required.	The expected workload is low-to-medium, and additional instances would increase operational complexity. Session storage would need to be externalised if multiple application instances are introduced.	Collect actual workload/resource evidence; observe memory, CPU and response behaviour; determine whether a single instance becomes insufficient; only then evaluate multiple instances and external session storage.	Deferred beyond the initial deployment unless M3 evidence demonstrates a need.

These decisions are deliberately deferred rather than unresolved by omission. The current M2 baseline establishes enough architecture, technology and deployment direction for implementation to proceed, while leaving provider-specific and operational decisions open until representative evidence exists. This is consistent with the existing PED baseline, which already defers the production host, attachment-storage architecture and automated deployment until later evidence is available

11. RTM & Traceability Foundation

11.1 Evolved RTM

According to the CivicConnect Master Project Brief, the team must maintain one evolving Requirements Traceability Matrix (RTM), with requirements linked from their stakeholder/source through later design, implementation, testing and acceptance/release evidence. The Milestone 1 brief requires the initial RTM to establish traceability from the stakeholder/source to the requirement and acceptance criteria, while providing a structure that can be expanded with later lifecycle evidence.

In this document, the source/stakeholder, requirement description, priority and acceptance criteria are already defined in the Functional and Non-Functional Requirements tables. The RTM therefore uses the Requirement ID as the link to these definitions and extends the traceability structure to future design/architecture, GitHub Issue/Pull Request, implementation, verification/testing and acceptance/release evidence. Fields relating to future milestones are marked as pending until the relevant controlled evidence is produced.

11.2 Requester RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-101	Request Management module; PostgreSQL service_requests + request_categories; Modular Monolith	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-102	Request Management + Audit / Request History; service_requests status tracking	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-103	Request Management + Audit / Request History; requester-linked request persistence	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-104	Observer / Event-Based Pattern (ADR-005); NotificationService; Nodemailer / SMTP	Pending - M3	emailService.js structural stub; EventEmitter logic Pending - M3	Pending - M3	Pending - M4
FR-105	Authentication & User Management module; ASR-01; users table; Node.js / Express security boundary (ADR-004)	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-106	request_attachments persistence design; external durable file storage; final storage provider deferred to M3	Pending - M3	Pending - M3	Pending - M3	Pending - M4

11.3 Staff RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-201	Assignment & Workflow module; service_requests assignment model + assignment history	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-202	Centralised Workflow FSM / Transition Service (ADR-006); RequestWorkflowService; status-history persistence	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-203	Search, Reporting & Dashboard module; indexed request filtering by status/category/assignment	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-204	Request Workflow + request_comments; resolution/action audit information	Pending - M3	Pending - M3	Pending - M3	Pending - M4

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-205	ASR-01 Controlled Access; Request Management + department/role authorisation boundary	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-206	Centralised Workflow FSM (ADR-006); RequestWorkflowService; resolution/closure persistence	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-207	ASR-01; Department-Based Access; staff_assignments + departments + request_categories (DEC-007)	Pending - M3	Pending - M3	Pending - M3	Pending - M4

11.4 Management RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-301	Search, Reporting & Dashboard module; service_requests + departments; status/overdue query design	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-302	Audit & Request History module; request_status_history, request_assignment_history, request_comments, admin_audit_log; ADR-005	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-303	Assignment & Workflow module; service_requests + request_assignment_history; ASR-02	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-304	Search, Reporting & Dashboard module; category/status/department persistence and indexed query design	Pending - M3	Pending - M3	Pending - M3	Pending - M4
FR-305	ASR-01 Controlled Access; management authorisation; contact_information_approved, contact_approved_by, contact_approved_at (DEC-007)	Pending - M3	Pending - M3	Pending - M3	Pending - M4

11.5 Engineering Decision Log

The Engineering Decision Log records significant decisions made during Milestone 1 so that their rationale, trade-offs, risks and later consequences remain traceable throughout the project. Where insufficient evidence currently exists to support a responsible decision, the decision is deliberately deferred and the information required before commitment is identified. In accordance with the CivicConnect Master Project Brief, significant decisions are recorded together with their context, constraints, alternatives, rationale, risks, supporting evidence and later consequences.

Field	Entry
Decision ID	DEC-001
Description	Requirement Identification Convention
Context	CivicConnect requirements must remain identifiable and traceable across the PED, RTM and later engineering artefacts. The team therefore needs a consistent naming and numbering convention.
Constraints	Requirements must use unique and stable identifiers and must remain traceable throughout later design, implementation, testing and change activities.
Alternatives	1. Use a single sequential numbering scheme for all requirements. 2. Separate Functional and Non-Functional Requirements and use numbering groups to indicate stakeholder or quality category.
Decision	Functional Requirements will continue to use stakeholder-based numbering, with FR-1xx for Requester requirements, FR-2xx for Staff requirements and FR-3xx for Management requirements. Non-Functional Requirements will use category-based numbering.
Rationale	The numbering structure allows the team to identify the primary stakeholder or quality category associated with a requirement while maintaining stable identifiers for traceability. As the scope now contains related behaviour across Requesters, Staff and Management, this separation also reduces ambiguity when requirements are later linked to design, implementation, testing and change evidence.
Trade-offs	The convention improves organisation and traceability but requires the team to maintain the numbering structure consistently as new requirements are added or changed.
Risks	1. Incorrect, duplicated or inconsistently referenced IDs could break links between the requirements tables, RTM, GitHub issues, later tests and release evidence.
Evidence	Functional Requirements table, Non-Functional Requirements table and RTM within the PED.
Later Consequence	The same requirement IDs will continue to be used in architecture/design artefacts, GitHub issues and PRs, implementation evidence, tests and acceptance evidence.

Field	Entry
Decision ID	DEC-002
Description	GitHub Branching and Review Strategy
Context	The team requires a controlled GitHub workflow that prevents uncontrolled changes to stable code and documentation while allowing three team members to work concurrently. During M2, automated CI and the need for controlled progression toward staging required the original M1 branching strategy to evolve.
Constraints	The main branch must remain highly protected. Changes must enter controlled branches through Pull Requests. However, requiring two independent reviewers for every development-level change creates a bottleneck for a three-person team.
Alternatives	1. Retain only main and dev with two approvals on all protected branches. 2. Retain main and dev but reduce approval requirements for dev (main remains unchanged). 3. Introduce testing and staging branches and apply stronger controls as code moves closer to release.
Decision	The project will use main, staging, testing and dev as controlled branches, together with temporary feature/documentation/fix branches. main and staging require 2 independent approvals. dev and testing require 1 independent approval. The testing branch additionally requires the CI status check to pass before merging.
Rationale	The revised approach preserves peer review while reducing collaboration bottlenecks during active development. Stronger controls are applied as code progresses toward staging and production.

Trade-offs	More branches increase branch-management and promotion overhead, while reduced approvals on dev and testing provide less manual review than main and staging.
Risks	Poor branch management may cause divergence, merge conflicts or confusion about which branch represents the current integration state. Reducing approvals also increases reliance on CI and effective peer review.
Evidence	GitHub branch rulesets, protected branch configuration, Pull Request history, Section 14.1 and CI evidence.
Later Consequence	M2 Amendment: The original M1 main + dev strategy has been expanded. The testing branch is now the current code-integration branch and requires one independent approval plus successful CI. The staging branch exists but its deployment integration remains deferred to M3.

Field	Entry
Decision ID	DEC-003
Description	Exclusion of High-Complexity Communication and Media Features
Context	CivicConnect could support additional interaction mechanisms such as an AI chatbot, internal real-time messaging, video uploads and voice input.
Constraints	The project is limited by schedule, team size, cost, storage capacity, security considerations and the requirement to prioritise the core service-request workflow.
Alternatives	1. Include all proposed communication/media features. 2. Include selected additional features. 3. Exclude them and focus on the core request workflow.
Decision	AI chatbot functionality, internal real-time messaging, video uploads and voice input remain excluded from the committed baseline.
Rationale	These features are not necessary to satisfy the core CivicConnect business need and would introduce additional implementation, storage, security, testing and maintenance complexity.
Trade-offs	Users receive fewer communication/input options, but the team can focus resources on the core request lifecycle, security, traceability and quality.
Risks	Some users may prefer richer communication mechanisms. Adding them later would increase scope and would require formal change analysis.
Evidence	PED Out-of-Scope section.
Later Consequence	Architecture, storage and interface design do not need to support video, audio, chatbot or internal real-time messaging functionality.

Field	Entry
Decision ID	DEC-004
Description	Defer Final Technology Stack and Architecture Selection
Context	During M1, CivicConnect had identified authentication, departmental routing, image attachments, dashboards, notifications, role-based access and responsive requester-facing interfaces, but insufficient evidence existed to responsibly commit to a final architecture and technology stack.
Constraints	Technology and architecture selection had to consider requirement fit, team capability, learning curve, cost, security, testing support, maintainability and deployment compatibility.
Alternatives	1. Select a preferred stack and architecture immediately in M1. 2. Make a provisional selection based mainly on familiarity. 3. Defer final selection until M2.
Decision	During M1, the final technology stack and architecture were deliberately deferred to M2.
Rationale	Deferring prevented premature commitment before requirements, ASRs and constraints were sufficiently established.
Trade-offs	Deferral preserved flexibility but reduced the time available for implementation once the choices had to be finalised.
Risks	Deferring too long could create schedule pressure, while deciding too early could result in an unsuitable solution.

Evidence	M1 PED scope, constraints, requirements and forward-engineering considerations.
Later Consequence	M2 Resolution: This decision is now resolved. CivicConnect has selected a Modular Monolith architecture and a JavaScript-centric MVC technology stack consisting of Node.js v24, Express.js, EJS, Tailwind CSS, PostgreSQL and node-postgres. See DEC-009, DEC-010 and DEC-011.

Field	Entry
Decision ID	DEC-005
Description	Defer CI / GitHub Actions Configuration
Context	During M1, the final build process, technology stack and testing framework had not yet been selected, so configuring CI at that stage would have relied on assumptions.
Constraints	The CI workflow must correspond to the actual runtime, dependency installation process, testing framework and quality checks used by CivicConnect.
Alternatives	1. Configure GitHub Actions during M1 using assumed technologies. 2. Create only a placeholder workflow. 3. Defer exact CI implementation until the technology stack was known.
Decision	During M1, CI implementation was deliberately deferred.
Rationale	This avoided configuring a workflow that might later require significant rework.
Trade-offs	Deferral delayed automated quality evidence but avoided throwaway implementation.
Risks	Delaying CI too long could allow defects or inconsistent code to enter integration branches without automated checks.
Evidence	M1 decision log and M2 CI implementation evidence.
Later Consequence	M2 Resolution: GitHub Actions is now implemented as the initial CI quality gate. Node.js v24 is pinned through .nvmrc; npm ci installs dependencies; ESLint/Prettier performs static checks; and Jest executes automated tests. The CI status check must pass before merging into testing. Production Continuous Deployment remains deferred to M3.

Field	Entry
Decision ID	DEC-006
Description	Use Email-Based Registration Rather Than Social Login
Context	CivicConnect requires user identity for authentication, requester ownership, staff/management roles, department membership and email notifications.
Constraints	External authentication providers introduce additional APIs, configuration, security considerations and testing requirements.
Alternatives	1. CivicConnect-managed email/password registration. 2. Third-party/social login. 3. Support both approaches.
Decision	CivicConnect will use an email-based registration and authentication mechanism. Social login will not form part of the committed baseline.
Rationale	Email is already required for notification functionality and provides a single controlled identity mechanism without an additional external identity-provider dependency.
Trade-offs	The team must manage credential security and password handling directly.
Risks	Poor authentication or password-management implementation could expose user accounts.
Evidence	Authentication Functional Requirement, Out-of-Scope section and NFR-101.
Later Consequence	Architecture and implementation must support secure credential handling, server-side authentication and role/account association.

Field	Entry
Decision ID	DEC-007
Description	Department-Based Request Access and Management-Controlled Contact Information
Context	CivicConnect must limit service-request visibility according to operational responsibility while protecting requester contact information.
Constraints	Requests may contain sensitive contact and location information. Staff should only access requests relevant to their department.
Alternatives	1. All Staff may view all requests. 2. Restrict requests by department but allow unrestricted requester-contact access. 3. Restrict requests by department and require Management approval for requester contact details.
Decision	Requests are made available only to the relevant department. Staff may accept responsibility only for eligible requests within that department. Requester contact information remains restricted until authorised by the relevant manager.
Rationale	This reduces unnecessary exposure of requester information while supporting operational ownership.
Trade-offs	The approach introduces additional permissions, approval logic and Management involvement.
Risks	Incorrect categorisation could route requests incorrectly. Incorrect access-control implementation could either expose requester information or block legitimate users.
Evidence	FR-207, FR-305, NFR-101 and the M2 persistence model.
Later Consequence	M2 data design now represents roles, departments, assignments, category routing and contact-information approval. M3 must implement and verify allowed and denied access cases.

Field	Entry
Decision ID	DEC-008
Description	Partial Mobile Responsiveness Scope
Context	Requesters are likely to submit and track requests from mobile devices, while Staff and Management screens may contain denser operational information.
Constraints	Full responsiveness across all roles would increase design and testing effort within the limited project schedule.
Alternatives	1. Make every interface fully responsive. 2. Target desktop only. 3. Prioritise Requester-facing responsive design and defer Staff/Management mobile scope.
Decision	Requester-facing functionality will support mobile browser use. The extent of Staff and Management mobile responsiveness remains deferred.
Rationale	Requesters have the strongest immediate need for mobile access, while Staff and Management interfaces are likely to require denser desktop-oriented layouts.
Trade-offs	Staff and Management may have a less optimised mobile experience.
Risks	Later evidence may show that Staff or Management require more mobile functionality, causing additional redesign.
Evidence	NFR-401, In-Scope Mobile Accessibility and Deferred Scope.
Later Consequence	Requester-facing responsive behaviour remains part of implementation. Staff and Management mobile scope must be revisited when their final workflows and interfaces are better established.

Field	Entry
Decision ID	DEC-009
Description	Select Modular Monolith Architecture

Context	CivicConnect requires clear responsibility boundaries for authentication, request processing, workflow, auditing, notifications and reporting, while being implemented by a three-person team under a limited schedule.
Constraints	The architecture must support ASR-01 to ASR-05, remain low-cost, avoid unnecessary deployment complexity and support maintainability and testing.
Alternatives	1. Traditional Layered Monolith. 2. Microservices / Distributed Architecture.
Decision	CivicConnect will use a Modular Monolith architecture. It will remain a single deployable application while major responsibilities are separated into internal modules.
Rationale	The Modular Monolith provides stronger internal separation than a basic monolith without introducing the network, deployment and data-consistency complexity of microservices.
Trade-offs	Individual modules cannot be independently deployed or scaled.
Risks	If internal boundaries are not maintained, the application could gradually become tightly coupled. A failure in the single application process may also affect the complete system.
Evidence	Architecture comparison, Sections 5.1-5.6, architecture diagrams and architecture ADR.
Later Consequence	M3 implementation must preserve the defined internal boundaries between presentation, request handling, business rules, persistence and supporting services.

Field	Entry
Decision ID	DEC-010
Description	Select PostgreSQL as the Relational Persistence Technology
Context	CivicConnect must reliably persist users, requests, assignments, workflow history, comments, audit information and departmental relationships.
Constraints	The persistence technology must support relational integrity, transactions, concurrency protection, efficient queries and low-cost deployment.
Alternatives	1. Relational PostgreSQL persistence. 2. A document-oriented database. 3. Simplified file/in-memory persistence.
Decision	CivicConnect will use PostgreSQL as its persistent relational database and node-postgres as the database access layer.
Rationale	PostgreSQL supports relational constraints, transactions, row-level locking and the structured relationships needed for CivicConnect's request lifecycle and access-control model.
Trade-offs	PostgreSQL introduces reliance on a managed database environment for production and requires appropriate schema and transaction design.
Risks	Poor transaction or concurrency handling may result in lost updates or inconsistent request state. Provider-specific storage and recovery capabilities may also be insufficient.
Evidence	Section 6 Data & Persistence Baseline, ERD/schema and technology-selection section.
Later Consequence	M3 implementation must connect the application to PostgreSQL and implement transaction and concurrency controls. The database technology is decided, but the managed provider remains deferred between candidates such as Neon and Supabase.

Field	Entry
Decision ID	DEC-011
Description	Select JavaScript-Centric MVC Technology Stack
Context	The project requires a practical web technology stack that the three-person team can implement and maintain within the milestone schedule while supporting authentication, database access, responsive pages, testing and deployment.
Constraints	The stack must remain maintainable, low-cost, lightweight enough for limited hosting environments and compatible with the team's capability.

Alternatives	1. React/Next.js SPA with separate API backend. 2. Java Servlets / Spring Boot. 3. JavaScript-centric server-rendered MVC application.
Decision	CivicConnect will use Node.js v24, Express.js, EJS, Tailwind CSS and node-postgres as its primary application stack.
Rationale	A unified JavaScript stack reduces learning and integration overhead. Server-side rendering avoids a separate frontend deployment and reduces unnecessary client-side state complexity.
Trade-offs	EJS provides a less dynamic client experience than a SPA architecture and generally relies more heavily on full-page server-rendered interactions.
Risks	Poor separation of responsibilities could still produce tightly coupled code even within an MVC structure. Runtime or dependency differences could also create environment inconsistencies if versions are not controlled.
Evidence	Section 7 Technology Stack Decision; ADR-004; .nvmrc; package.json; application repository.
Later Consequence	Node.js v24 is pinned through .nvmrc. M3 implementation will continue using the selected Express/EJS structure and PostgreSQL integration rather than introducing a separate SPA frontend.

Field	Entry
Decision ID	DEC-012
Description	Decouple Core Request Transactions from External Side-Effects
Context	Status changes and request-resolution operations must update persistent CivicConnect state but may also trigger secondary behaviour such as email notifications and audit processing.
Constraints	Failure or delay in an external service such as SMTP must not cause a valid request-state transaction to fail or remain open unnecessarily.
Alternatives	1. Direct synchronous service orchestration. 2. Observer / Event-Based internal side-effect handling.
Decision	CivicConnect will use an Observer / Event-Based Approach for secondary side-effects. RequestWorkflowService commits the primary transaction first, then publishes an in-memory event which supporting listeners such as NotificationService and AuditLogService respond to.
Rationale	This keeps the core request state independent from slower or less reliable supporting functions.
Trade-offs	In-memory events are lightweight but are not durable across application-process failure.
Risks	An event may be lost if the Node.js process fails after the database commit but before listeners complete. Notification failures may therefore occur even though the underlying request state is correct.
Evidence	Initial Design Decision 1; ADR-005; src/services/emailService.js structural stub; RSK-013.
Later Consequence	M3 must implement the EventEmitter-based behaviour and ensure that email/audit failures do not roll back valid request transactions. The platform remains the source of truth even if a notification fails.

Field	Entry
Decision ID	DEC-013
Description	Centralise Request Lifecycle State Mutation
Context	The request lifecycle includes controlled transitions such as SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED and CLOSED. Duplicating transition logic across controllers would increase inconsistency and security risk.
Constraints	Workflow rules must remain maintainable, testable and consistently enforced across the application.
Alternatives	1. State Pattern with separate state classes. 2. Procedural Centralised Workflow / Transition Service.

Decision	Request lifecycle changes will be controlled through a single RequestWorkflowService containing a central allowed-transition configuration / finite state machine. Controllers will delegate mutation requests to this service.
Rationale	The centralised service provides consistent transition validation without the additional boilerplate of separate state classes for a relatively small fixed lifecycle.
Trade-offs	The service may become too dense if too many responsibilities are added.
Risks	A poorly structured central service could become a new monolithic component.
Evidence	Initial Design Decision 2; ADR-006; src/services and src/controllers structure.
Later Consequence	Validation rules should be extracted where appropriate so that RequestWorkflowService remains an orchestrator rather than accumulating unrelated logic. All request state mutations in M3 must pass through this controlled boundary.

Field	Entry
Decision ID	DEC-014
Description	Select Nodemailer with SMTP as the Application-Level Email Integration
Context	FR-104 requires requesters to receive feedback when request status changes. The email mechanism must integrate with the Node.js application without creating unnecessary architectural complexity.
Constraints	The solution should remain low-cost, configurable across environments and independent from the core request transaction.
Alternatives	1. Managed REST-based transactional email API. 2. Broker-based asynchronous messaging. 3. Direct SMTP integration using Nodemailer.
Decision	CivicConnect will use Nodemailer with SMTP as its application-level email integration mechanism. Ethereum will be used for development/testing.
Rationale	Nodemailer integrates naturally with Node.js, introduces minimal vendor-specific application logic and allows SMTP configuration to vary between environments.
Trade-offs	Direct SMTP depends on hosting-provider networking restrictions and SMTP-provider reliability.
Risks	Current deployment research shows that Render Free blocks outbound SMTP and Railway restricts SMTP to higher plans. Therefore the current production SMTP direction may conflict with low-cost hosting constraints.
Evidence	Section 9 Initial API / Integration Decisions; ADR-005; deployment compatibility research; RSK-008 and RSK-013.
Later Consequence	The application-level mechanism is selected, but the final production email provider/configuration remains deferred to M3. The production solution must be proven compatible with the selected host.

Field	Entry
Decision ID	DEC-015
Description	Establish Deployment Environment Progression and Single-Instance Baseline
Context	CivicConnect must progress from local development toward production without assuming that behaviour proven locally or in CI automatically proves production compatibility.
Constraints	The team has limited infrastructure capacity and must avoid premature commitment to a paid or incompatible cloud environment.
Alternatives	1. Deploy directly from development to production. 2. Introduce a containerised/distributed deployment pipeline immediately. 3. Progress through controlled environments before production commitment.

Decision	CivicConnect will use the progression Development -> Test/CI -> Local Staging -> Production. The current deployment baseline is a single application instance, consistent with the Modular Monolith architecture.
Rationale	Local staging allows the team to validate the integrated application, configuration, database connectivity and supporting services before selecting and committing to production infrastructure.
Trade-offs	This introduces an additional staging step and delays final cloud-provider selection.
Risks	Environment-specific configuration differences may cause behaviour that works in development or CI to fail in staging or production.
Evidence	Section 10 Deployment Compatibility, RSK-014 and Section 15 Architecture, Technology and Initial Design Baseline.
Later Consequence	Render and Railway remain candidate production application hosts, while Neon and Supabase remain candidate PostgreSQL providers. Final hosting, database provider, attachment storage, production email configuration, production secrets, Continuous Deployment, monitoring, recovery validation and horizontal scaling remain deferred until M3 evidence is available.

Please note that due to the space constraints in Microsoft Word the format in which the Engineering Decision Log is displayed in this PED is not the same as the table format we will record and work with in the Engineering_Decision_Log.md file in our GitHub repository.

12. Risk & Forward Engineering Considerations

12.1 Initial Risk Register

In accordance with the CivicConnect Master Project Brief, the Risk Register is maintained as a live engineering artefact and records project-specific risks together with their cause, probability, impact, priority, mitigation, contingency, owner and current status. Risks are identified from the current CivicConnect scope, requirements, constraints and engineering decisions and will be reviewed and updated as new evidence becomes available throughout subsequent milestones. Where a risk occurs, it will be treated as an issue and linked to the relevant engineering actions or decisions.

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-001	Schedule overrun due to scope and learning curve The team may fail to complete the committed CivicConnect scope within the required milestone schedule.	The project contains multiple user roles, authentication, departmental access, dashboards, notifications, image uploads, responsive requester-facing functionality, database integration and deployment work, while the team consists of only three members and must complete the remaining implementation within a limited milestone schedule.	Medium	High	Critical	Maintain the baselined scope; prioritise Must-have requirements; use the selected JavaScript/Node.js stack to avoid unnecessary learning overhead; track milestone work through GitHub issues; keep work visible through PRs; avoid implementing deferred or out-of-scope features without formal change analysis.	If schedule variance becomes significant, defer or remove lower-priority Should/Could requirements through controlled change rather than reducing essential testing, security or quality controls.	Development Team Product Owner	Open - Active

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-002	Uncontrolled scope growth Additional feature requests or ideas may expand the committed project scope and threaten delivery of the baseline requirements.	The Product Owner may want additional functionality, and the team has already identified several deferred features that could appear valuable later.	Medium	High	High	Maintain the approved baseline scope; clearly distinguish in-scope, out-of-scope and deferred functionality; require formal change requests and impact analysis before accepting additional features; update affected requirements, RTM, decisions and risks through controlled change.	Reject or defer proposed changes where schedule, cost, quality or security impact cannot be accommodated. If a change is necessary, update the controlled artefacts before implementation.	Product Owner Development Team	Open - Active
RSK-003	Unauthorised access to requester information Requester contact information or service-request data may be exposed to users who are not authorised to access it.	CivicConnect has Requester, Staff, Management and Admin roles, department restrictions, staff assignment rules and management-controlled access to requester contact details, creating complex server-side authorisation requirements.	Medium	High	Critical	Implement authentication, secure password hashing, server-side RBAC and department checks; centralise access-control rules; restrict requester contact information by default; use secure session configuration; test permitted and denied scenarios; keep secrets out of source control.	If an access-control flaw is found, restrict or disable affected functionality, correct the authorisation logic, investigate any exposed information and verify the fix before restoring access.	Development Team Management	Open - Active
RSK-004	Incorrect request categorisation or departmental routing A service request may be categorised incorrectly and therefore become unavailable to the department responsible for resolving it.	Requesters select a category when creating a request, and the category determines which department can access and handle the request.	Medium	High	High	Use a controlled category list with clear labels/descriptions; validate category-to-department mappings; allow Management to redirect or reassign requests where necessary; test department access rules.	Allow Management to correct or redirect incorrectly categorised requests and record the correction through the audit/history trail.	Requester Management Development Team	Open - Active
RSK-005	Unsuitable production deployment platform or provider selection The selected production platform may not adequately support CivicConnect's operational requirements and constraints.	The application architecture and technology stack are now selected, but the final PaaS host and managed PostgreSQL provider remain deferred. Candidate providers differ in memory, free-tier limits, SMTP support, database pooling, backup/recovery and operational behaviour.	Medium	High	High	Keep final provider selection deferred until M3 evidence exists; validate the integrated Node.js application on representative infrastructure; compare Render/Railway and Neon/Supabase against runtime support, PostgreSQL connectivity, secrets/configuration, SMTP compatibility, availability, recovery and total operating cost.	If a provider proves unsuitable, stop production release and move to the next evaluated candidate or adjust the supporting service configuration before substantial production dependence is created.	Development Team	Open - Decision Pending M3

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-006	Free-tier resource or service limitations CivicConnect may exceed the limits of free-tier hosting, database, storage, CI or supporting services.	The project must remain free or low-cost, while application hosting, PostgreSQL, image storage, email delivery and CI consume limited resources. Current provider research shows material quota and plan restrictions.	Medium	High	High	Monitor resource consumption and provider quotas; measure application memory and database usage; use connection pooling; enforce image limits; keep binaries out of the application instance; compare actual monthly cost before production; select low-cost alternatives only where they still satisfy required functionality.	Reduce non-essential resource consumption, tighten upload limits, move to another evaluated provider or accept a justified paid tier through controlled decision if the free option cannot meet the baseline.	Development Team Product Owner	Open - Monitoring
RSK-007	Image upload creates storage or security problems User-uploaded images may consume excessive storage or introduce unsafe or unsupported content.	Image uploads are in scope, while the final file-size limit, storage provider and complete validation approach are still deferred to M3.	Medium	Medium/High	High	Allow only supported image types; validate file type and size server-side; enforce upload limits and authorisation for retrieval; store metadata and stable file references; monitor storage quotas; test the final storage mechanism before production.	Temporarily reduce supported file sizes/types or disable FR-106 upload functionality until storage/security constraints are corrected and verified.	Development Team Requester	Open - Decision Pending M3
RSK-008	Email notification dependency or delivery failure Requesters may not receive status notifications reliably even when the underlying request is updated correctly.	Email delivery depends on Nodemailer plus the final SMTP provider, host networking rules, provider sending limits, authentication and external service availability.	Medium	Medium	Medium	Keep notification processing separate from the core request transaction; log delivery success/failure; test with Ethereum during development and the final provider in staging/production; keep credentials externalised; allow requester notification preferences; verify host SMTP compatibility before production.	Retain the correct status within CivicConnect if email fails; allow users to view status in the platform; retry or manually resend where appropriate; switch provider/configuration if required.	Development Team Requester	Open - Monitoring
RSK-009	Team-member availability or knowledge concentration Progress may be delayed if important project knowledge or work becomes concentrated in one team member who becomes unavailable or falls behind.	The project is completed by only three team members and members may become primary contributors to specific technical or documentation areas.	Medium	High	High	Maintain readable documentation and code; require peer review; ensure more than one member understands major requirements, decisions and artefacts; use GitHub issues/PRs to keep work visible; avoid isolated ownership of critical areas.	Reassign affected work using documented artefacts, GitHub history and peer-reviewed contributions; reduce lower-priority work if necessary to protect milestone delivery.	Development Team	Open - Active
RSK-010	Loss or corruption of platform data Service-request records, assignment information, status history, audit information or other persistent data may be lost, corrupted or become inconsistent.	PostgreSQL is selected, but the final managed provider and verified backup/recovery process are still deferred. Provider free tiers differ in backup/restore capability, and recovery has not yet been proven.	Low/Medium	High	High	Use PostgreSQL integrity constraints and transactions; validate writes; maintain audit/history records; keep database persistence independent of the application instance; target RTO 8 hours and RPO 24 hours; select a provider that can support recovery needs; perform at least one restore test before production.	Pause production writes if corruption is detected; restore from the latest verified backup/recovery point; reconnect the application through updated configuration; run smoke tests before resuming service; prevent release if recovery cannot be demonstrated.	Development Team Management	Open - Monitoring

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
New Risks identified in M2									
RSK-011	Lost update / inconsistent request state due to unresolved concurrency control Two users may modify the same request at the same time and one change may overwrite another.	Staff and Management can perform overlapping operations such as status updates, assignment/reassignment and contact-access approval. The final concurrency mechanism is still to be proven during M3.	Medium	High	High	Treat concurrency control as required; retain a version column on service_requests; evaluate optimistic locking against SELECT FOR UPDATE; wrap status transitions in atomic transactions; create simultaneous-update tests.	Reject stale/conflicting updates, ask the user to refresh and retry, and correct affected records using request_status_history and request_assignment_history where necessary.	Development Team	Open - Decision Pending M3
RSK-012	Attachment storage is undecided and images may be lost or unavailable Uploaded images may be stored non-durably, exceed provider limits or delay FR-106.	The PED requires durable attachment storage, but the final mechanism/provider is deferred. Local application-instance storage is not durable across restart or redeployment.	Medium	High	High	Store only metadata and a stable file reference in request_attachments; do not rely on application-instance storage for production; prototype a durable external-storage approach early in M3; enforce file size/type limits; verify access control, retention, recovery and cost.	Temporarily disable or limit FR-106 until a durable storage option is confirmed; migrate any temporary/local files to the selected durable storage before production.	Development Team	Open - Decision Pending M3
RSK-013	Email notification failure or silent loss due to in-memory events Notifications may be lost or duplicated even though the core request update succeeds.	The selected Observer/Event-Based approach uses an in-memory Node.js event emitter for secondary actions. Events do not survive a process crash, while SMTP delivery also has external failure modes.	Medium	Medium	Medium	Log all notification outcomes; assign a notification/event identifier to reduce duplicates; attempt a controlled retry for transient failures; test with Etherscan and the final SMTP provider; ensure the platform remains the source of truth for request status.	Mark failed notifications for audit/manual retry; allow the requester to view the correct status inside CivicConnect; switch SMTP provider/configuration if needed.	Development Team	Open - Monitoring
RSK-014	Deployment environment or configuration mismatch CivicConnect may work in development or CI but fail in local staging or production because environment-specific configuration differs.	The deployment path spans Development -> Test/CI -> Local Staging -> Production. Database URLs, SESSION_SECRET, SMTP credentials, NODE_ENV and provider-specific settings are externalised and may be missing, invalid or inconsistent between environments.	Medium	High	High	Pin Node.js v24 through .nvmrc; use package-lock.json/npm ci; document all required environment variables; keep secrets external to source control; create consistent environment templates; test startup with missing/invalid configuration; validate the integrated application in local staging before production.	Block deployment, compare the affected environment with the validated staging configuration, correct the mismatch, rerun health/smoke tests and only continue once configuration parity is restored.	Development Team	Open - Monitoring

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-015	Insufficient verification / test coverage before integration or release Defects may pass the current quality gate because important functionality is not yet covered by automated or systematic tests.	At the end of M2, automated verification is mainly the /health test. Authentication, workflow, database operations, concurrency, UI interactions, attachments and email behaviour are not yet fully implemented or tested.	Medium	High	High	Expand Jest unit/integration tests during M3; add tests for authentication/authorisation, request workflow, database transactions, validation, concurrency and notification failure; add end-to-end and responsive/browser checks where practical; map tests back to the RTM and acceptance criteria.	Prevent release where critical requirements lack verification evidence; prioritise tests for Must-have/security/data-integrity paths; fix failed behaviour and rerun the full relevant test set before promotion.	Development Team	Open - Active

Forward Engineering Considerations

The considerations below identify why each concern is relevant to the current engineering baseline, which later decisions or activities it may influence, what information is still required before a responsible decision can be made, and the potential consequence of ignoring the concern.

Security, Authentication and Access Control

Security and access control already influence the CivicConnect baseline because the system will store requester information and provide different levels of access to Requesters, Staff and Management. Requests are also restricted according to department, while requester contact information may only be released to Staff after Management approval. This means that later architecture and implementation decisions must support controlled access rather than treating all authenticated users in the same way. The current requirements already establish department-based access and management-controlled contact information.

- **Later decision/activity influenced:** Authentication architecture, role-based access control, department-based permissions, database relationships, API access rules, security testing and deployment configuration.
- **Information still missing:** Final authentication technology, detailed role and permission structure, session or token mechanism, password-storage approach and the technical method used to represent Management approval of contact-information access.
- **Risk if ignored:** Security may be introduced too late in the design process, which could require significant redesign or result in unauthorised users gaining access to requests or requester information

Data Persistence, Integrity, Auditability and Recovery

Reliable data storage is very important when it comes to the CivicConnect project because currently the operational processes suffer from requests being lost, duplicated, overlooked or poorly tracked. The new system must therefore preserve request information, assignments, status changes, resolution details and lifecycle history reliably.

- **Later decision/activity influenced:** Database technology, schema design, relationships, transaction handling, audit logging, backup and recovery planning.
- **Information still missing:** Final database technology, persistence model, audit fields, retention requirements, backup frequency and recovery approach.
- **Risk if ignored:** Poor data design could recreate the original business problem by allowing request records or accountability information to become incomplete, inconsistent or unrecoverable.

Testability, Verification and CI Quality Gates

The requirements and acceptance criteria defined during M1 will determine whether the team can later prove that CivicConnect works correctly. The current PED already requires measurable evidence, automated testing and Continuous Integration rather than relying only on claims that the code works.

- **Later decision/activity influenced:** Test strategy, unit/integration/system testing, testing framework selection, GitHub Actions configuration, merge quality gates and release evidence.
- **Information still missing:** Final technology stack, testing tools, build process, CI workflow, performance-test conditions and quality thresholds.
- **Risk if ignored:** Requirements may be implemented without reliable verification evidence, while vague acceptance criteria could become difficult to test later.

Deployment, Storage, Free-Tier Limits and Operational Cost

CivicConnect must operate within free or low-cost constraints, but the final hosting, database and storage environment has not yet been selected. Features such as image uploads, persistent data, email notifications and authentication may all consume limited platform resources. The PED already identifies free-tier limits as an engineering constraint.

- **Later decision/activity influenced:** Hosting platform, database service, image-storage approach, email service, deployment pipeline and operational-cost decisions.
- **Information still missing:** Storage quotas, database limits, bandwidth limits, image-size limits, expected usage volumes, email-service restrictions and final deployment platform.
- **Risk if ignored:** The selected solution may work during development but exceed free-tier limits, create unexpected costs or require late redesign of storage and deployment components.

Responsive UI and Stakeholder-Specific Mobile Use

Requester-facing mobile accessibility is already part of the baseline, while the extent of mobile responsiveness for Staff and Management has deliberately been deferred. This means the team must support requester mobile use without prematurely committing to making every interface fully responsive.

- **Later decision/activity influenced:** Wireframes, responsive layouts, browser support, UI architecture, usability testing and refinement of NFR-401.
- **Information still missing:** Staff and Management mobile-use requirements, final dashboard complexity, target screen sizes and supported browser/version range.
- **Risk if ignored:** The team could either overcommit to full mobile responsiveness and increase development effort unnecessarily or create requester-facing screens that are difficult to use on mobile devices.

13. Development and Application Evidence

This section records the actual state of the CivicConnect application at the end of Milestone 2.

13.1 Current Application Status

CivicConnect is currently an EJS-based navigational prototype. The application can render pages and navigate between them, but it has no working backend business logic. Forms such as “Add Staff” or “Add Request” display correctly but do not submit or process data. No database connection is active. A database setup script exists in the repository, but it has not been executed against a live PostgreSQL instance. The application is therefore a static, navigable frontend mock-up that demonstrates the intended page structure and user flows.

We have not yet deployed to staging as the staging environment as not yet been configured, this is deferred to M3, but the current code is on our testing branch: https://github.com/Waldo-Blom/SEN381_Bugbusters/tree/testing

13.2 Repository / Module Structure

- src/ – Main application source code.
 - app.js, server.js - Express application setup and server entry point.
 - config/database.js - Database configuration placeholder; not yet connected.
 - controllers/ - Route handlers for auth, manager, requester, staff, and general pages. These currently render views or contain stubs, not full business logic.
 - routes/ - Express routers that map URLs to controllers.
 - middleware/authMiddleware.js - Placeholder for future authentication and access control.
 - services/emailService.js - To be used for future SMTP/email notification integration.
 - data/mockData.js - Static mock data used to render views during the prototype phase (will be removed once the database has been implemented).
 - database/CivicConnect_BugBusters.sql - SQL schema setup script based on the ERD in Section 6.2.
 - models/ - To be used for future data models; currently empty.
 - utils/nav.js - Navigation helper.
 - views/ - EJS templates. Contains layouts/, pages/ organised by role (auth, requester, staff, manager), and partials/ for shared components such as headers, footers, request tables, and filters.
 - public/ - Static assets: CSS styles, and placeholder folders for images and client-side JavaScript.
- tests/ - Jest tests. Currently only health.test.js.
- .github/ - GitHub issue templates and the CI workflow (workflows/ci.yml).
- docs/ - Project documentation, including PED versions, registers (AI usage, decision log, risk register, RTM), governance, requirements, scope, and stakeholder analysis.

- **Root-level files** - package.json, package-lock.json, .nvmrc, ESLint/Prettier configs, jest.config.js, README.md.

This structure supports separation of concerns and provides clear places for future implementation, but most business-logic folders currently contain placeholders.

13.3 Initial Implementation Evidence

The primary implementation evidence for M2 is the EJS view layer. Templates exist for authentication, requester, staff, and manager pages, and they are wired through Express routes and controllers to render correctly.

Shared layouts and partials provide consistent navigation and page structure. The CI workflow (.github/workflows/ci.yml) runs ESLint and Jest on the staging branch. A health check test verifies that the Express application boots and the /health endpoint returns 200 OK. This demonstrates that the application runtime and basic CI quality gate are functional.

13.4 Database / Migration Evidence

The database schema is defined in src/database/CivicConnect_BugBusters.sql.

This script contains CREATE TABLE statements for the core entities and relationships described in the ERD (Section 6.2). It is version-controlled and represents the planned persistence layer. However, no live database connection has been established, and the script has not been applied to a running PostgreSQL instance. There is no migration history, no seeded data, and no active database access layer. src/config/database.js is present but not yet configured for real use.

13.5 Testing / Verification Evidence

Testing is currently limited to the health check. tests/health.test.js verifies that the Express application starts and returns a successful response from /health.

The CI pipeline runs this test automatically on pushes and pull requests targeting the staging branch. Branch protection requires the CI check to pass before merging, and the team has demonstrated that a failing test blocks a merge. There are no tests for authentication, request submission, workflow, database operations, or UI interactions because those features are not yet implemented. The health check forms the first traceable verification link in the RTM.

13.6 What is still left (TODOs)

- **Implement backend business logic** - Move controllers and routes beyond rendering views so they process data and enforce application rules.
- **Connect the database** - Configure src/config/database.js, run CivicConnect_BugBusters.sql, and establish a working PostgreSQL connection pool.
- **Make forms functional** - Enable request submission, staff registration, and other forms to submit and persist data.
- **Implement authentication** - Add real session handling, credential validation, and role-based access control.
- **Implement request workflow** - Add assignment, status transitions, comments, and audit history.
- **Implement email notifications** - Replace the emailService.js stub with working SMTP integration.

- **Implement image uploads** - Add attachment handling and finalise file-size and storage decisions.
- **Implement concurrency control** - Add optimistic concurrency control for request state changes.
- **Extend responsive UI** - Complete Requester-facing mobile support and review Staff/Management mobile needs.
- **Implement a mobile navigation menu** – The requester screen is mobile friendly in terms of the way that the main content displays on the screen, however a mobile “hamburger” navigation menu still needs to be implemented.
- **Make changes to EJS views** - most of the EJS views are already completed, however the Admin page where Managers and Staff users are managed / registered still needs to be created. Also, the current EJS pages are not final and might still be updated in M3.
- **Extend CI pipeline** - Add automated deployment, coverage thresholds, and end-to-end tests.
- **Populate empty folders** - Add data models, utilities, and other placeholder areas as implementation begins.

In milestone 3, the relevant issues on the GitHub repository will be created for each of the functional and non-functional requirements.

14. GitHub Governance

Link to our repo: https://github.com/Waldo-Blom/SEN381_Bugbusters

GitHub is a tool but just because you make use of GitHub does not mean that you applied proper software engineering. In order to apply software engineering, you need to apply software engineering practices. The following below describes exactly how we will use GitHub throughout the project to ensure these best practices.

14.1 Branches

We will have two main branches that we will use `main` and `dev`. From these two branches we will branch out further into subbranches referred to as feature branches.

M2 review: After research conducted in assignment 3, we introduced a testing branch to run our automated CI checks. All current code resides on testing. We also created a staging branch, but its implementation is deferred to M3 as it depends on our deployment environment. The staging branch currently contains only documentation, though branch protection rules are already in place. main also currently contains only documentation. All code is on testing.

According to our project brief the main branch should be treated as a fully protected branch that does not allow commits on the branch unless through a pull request and detailed review. We achieve this through applying a ruleset to the `main` and `dev` branch with the following settings:

- **Requires a pull request before merging:** All commits must be made to a non-protected branch and submitted via a pull request before they can be merged into main. Direct development on main it is therefore not possible.
- **Requires approvals:** Pull requests targeting main require a minimum number of approvals. We have set the required number of approvals before merging to 2 thereby matching the project master brief.
- **Force pushes are blocked:** The "Allow force pushes" setting is disabled, so force pushes to main are not permitted.

- **Deletions are blocked:** When the "Allow deletions" setting is disabled, the main and dev branch can't be deleted (Just to avoid accidental deletion of our production and development branches)
- **Do not allow bypassing the above settings:** This setting just makes sure no matter the role that the above-mentioned settings can be bypassed by admin privileges or roles.

M2 review: The team identified a collaboration risk: requiring 2 independent reviewers on a 3-person team could create bottlenecks if either reviewer is unavailable, potentially stalling progress and leading to larger, riskier merges. The dev branch approval requirement has therefore been adjusted to 1 independent approval to mitigate this risk while maintaining peer review and code quality. The same applies for the testing branch. The staging branch however follows the same rules as main requiring 2 independent reviews.

The testing branch follows the same protection as dev (1 independent approval) and additionally requires the CI status check to pass before merging. The staging branch has branch protection rules in place, but CI integration will be completed in M3 when deployment is established, along with the configuration of the "require deployments to succeed before merging" rule.

For the full ruleset of our repo please see: https://github.com/Waldo-Blom/SEN381_Bugbusters/settings/rules

Let's take a practical example:

Waldo was assigned the following task to complete: "Documentation for "Problem and business need"". Now he has to go and make a change on the `problem-and-business-need.md` file (stored in the docs folder). Instead of just working on the branch `main` (which is against the project spec and not good software engineering practices (as you would never develop straight on the live version of company software)), he instead creates an entire new branch called `docs/waldo-m1-artifacts`. He then in his own local environment makes changes to that mentioned .md file, either through single or multiple commits (whatever fits his use case).

Note: We use naming prefixes (like docs/ and feature/) to categorize the type of work. GitHub utilizes these prefixes by allowing us to filter branches by name, making it easy to see all documentation branches or all feature branches at once, this helps to improve navigation and organization in the repository (Github, 2025).

Here are other examples of feature branches from our naming convention:

- docs/ai-usage-register-m1-waldo
- feature/NFR-improve-login-loading-time
- feature/FR-implement-login

As the project evolves, we might add additional branching prefixes, we will identify where certain changes align and agree on a prefix for that code change type. When a new one is added we will also update it in the `Github-governance.md` file, under the docs folder of our repo.

Note branches get deleted after each milestone, this is done to keep the repo clean and structured, so we don't end up with for example 70 feature branches that are no longer used. But before deleting a feature branch it is first merged with the relevant branch either `main`, `dev` or `testing` depending on the scenario and then that branch absorbs the feature branch commits and changes through the PR, nothing is lost and the commit history is kept, that branch just can be accessed anymore but this is fine.

See our current branches here: https://github.com/Waldo-Blom/SEN381_Bugbusters/branches

14.2 Pull requests (PR's) and Code review process

Marco and Christian then first have to review this change within the pull request, treating the Pull request (PR) as a place to do a code review, meaning they look at Waldo's proposed changes and then comments and gives feedback. After both Marco and Christian approve this change made by Waldo (Waldo can't approve his own changes, he only verifies that his work is indeed correct before opening a pull request). This applies for all members, the scenario above is just for demonstrative purposes.

As per our Ruleset, the main branch requires at least 2 approvals (enforced automatically). We do not allow self-approval on any branch that requires reviews. Any merge conflicts are then resolved (if there are any).

Example PR: https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/2

14.3 Issues

When referring to issues in GitHub we don't necessarily mean it in terms of a software bug (Github, 2025). This refers to a specific task that has to be completed (Github, 2025). For our project we start by identifying what the deliverables are for a specific milestone and then assign we distribute the workload and assign specific tasks to specific people. But just because one person is in charge of a specific part of the workload does not mean that person is the only one working on that, we look at what that person has done and then review it and incorporate our feedback.

Example issue: https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/3

14.4 Milestones

The milestones on our GitHub page relate to the actual milestones of this project i.e. Milestone 1 on GitHub consists of the issues that are related to Milestone 1 project deliverables.

Every issue must be assigned to only one milestone (M1, M2, M3, or M4). Assigning issues to milestones allows us to track our overall progress, the milestone view automatically shows what percentage is complete (based on closed vs. open issues) and helps us see if we are on track or falling behind.

Link Milestones: https://github.com/Waldo-Blom/SEN381_Bugbusters/milestones

14.5 Labels

We use labels in our PR's and issues just to mark issues more clearly, for example:

- By priority:
 - `priority: Urgent` – Blocking issues, critical bugs etc
 - `priority: High` - important but not a blocking issue
 - `priority: Medium` - normal priority
 - `priority: Low` - Nice to have, can defer
- By the task type:
 - `Feature` - New functionality added
 - `Documentation` - Improvements or additions to documentation
 - `Bug` - Something isn't working

- `Enhancement` - Improve something / change an existing behaviour
- By the type of work:
 - `Front end` - UI / Client side changes
 - `Back end` - Server / API work
 - `Database` - Data layer / queries

In the future as the project evolves, we might add additional labels as we see fit in order to provide extra clarity where necessary. Or we might remove some that are not used.

Link to the labels: https://github.com/Waldo-Blom/SEN381_Bugbusters/labels

14.6 CI / Automated Checks

For M2, we implemented a lightweight Continuous Integration (CI) pipeline using GitHub Actions. This pipeline runs automatically on every push and pull request targeting the testing branch and currently performs the following steps:

1. Checks out the repository.
2. Sets up Node.js using the version pinned in .nvmrc (Node 24), ensuring consistent runtime across developer machines and CI.
3. Installs dependencies using npm ci, producing a clean, reproducible install from package-lock.json.
4. Runs ESLint with Prettier integration to enforce code quality and formatting rules across all .js files.
5. Runs the Jest test suite to execute automated tests.

The test suite currently contains a health check test (tests/health.test.js) that verifies the Express application boots and that the /health endpoint returns 200 OK. This provides initial automated verification that the application is functional and forms the first traceable link from requirement to verification evidence in the RTM.

Linting is deliberately scoped to JavaScript files only. ESLint does not natively parse .ejs templates, so src/views/ and src/public/ are excluded via .eslintignore. Template-level linting was assessed and deliberately deferred to M3, where a dedicated EJS syntax checker may be introduced.

Integration with branch protection

The CI pipeline is integrated with branch protection on testing: the status check CI / build-and-test must pass before a pull request can be merged. As per the M2 review in Section 14.1, the testing branch requires 1 independent approval (same as dev), and the CI check is a mandatory requirement in addition to peer review. This ensures that no code failing lint or tests can be merged into testing.

Secret handling

No secrets are committed to the repository. The SESSION_SECRET required by express-session is injected into the CI environment using GitHub Actions repository secrets. Production configuration of this secret is deferred to M3, where deployment environment variables will be formally established.

Scope and deliberate exclusions

The pipeline is intentionally minimal to meet M2 requirements without over-engineering. It does not include deployment, coverage thresholds, end-to-end tests, containerisation, or production observability. These are explicitly listed in the M2 brief as out of scope and are deferred to M3. In M3, the team plans to extend the pipeline to include automated deployment to a staging environment,

broader test coverage, and additional quality gates as the project matures. Consideration will also be given to extending CI coverage to the main branch once a production release process is established.

Evidence and traceability

The workflow configuration is available at `.github/workflows/ci.yml`, and the initial test is located at `tests/health.test.js`.

Evidence of the CI gate blocking a failing change:

https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/47

This PR was opened with a deliberately failing test to demonstrate that the branch protection rule and required status check are functioning as configured. GitHub blocked the merge as expected. The PR will be closed without merging, confirming that no failing code could enter testing.

15. Architecture, Technology and Initial Design Baseline

15.1 Baseline Identification

- Baseline: CivicConnect PED v2.0
- Milestone: M2
- Status: Approved for controlled construction

15.2 Baseline synthesis and operational contract

The CivicConnect PED v2.0 baseline establishes the engineering guardrails that will guide controlled implementation during Milestone 3. The baseline defines the architecture, persistence technology, application technology stack, initial design responsibilities, integration mechanism, verification approach and current deployment direction while deliberately retaining provider-specific operational decisions until sufficient implementation and staging evidence exists.

- **Architecture baseline:** CivicConnect uses a Modular Monolith deployed as a single application. Internally, responsibilities are separated through clear presentation, controller/request-handling, service/business-logic and persistence boundaries.
- **Data and persistence baseline:** PostgreSQL is the selected relational persistence technology. Transaction handling, integrity constraints and concurrency protection will preserve request and workflow consistency. The final managed PostgreSQL provider remains deferred.
- **Technology baseline:** CivicConnect uses Node.js v24, Express.js, EJS and Tailwind CSS, with node-postgres for database access. Node.js v24 is pinned through `.nvmrc` to support runtime consistency between development and CI.
- **Initial design baseline:** Core request workflow and persistence remain separated from secondary external side-effects. Notification activity must not cause an otherwise valid request transaction to fail.
- **Integration baseline:** Nodemailer using SMTP is the selected application-level email integration mechanism. Ethereal will be used for development/testing. The final production SMTP provider/configuration remains deferred because it must be verified against the networking restrictions of the selected production host.
- **Verification and CI baseline:** GitHub Actions, Jest and ESLint provide the initial automated quality gate. The staging branch requires peer review and a successful CI workflow before changes can progress.
- **Deployment baseline:** The current environment progression is Development → Test/CI → Local Staging → Production. CivicConnect will initially be validated as a single application instance in a dedicated local staging environment before moving to a managed PaaS production environment.

- **Production infrastructure baseline:** Render and Railway remain candidate application-hosting providers, while Neon and Supabase remain candidate managed PostgreSQL providers. Final selections are intentionally deferred until M3 evidence confirms compatibility, resource usage, cost, availability, backup/recovery and operational suitability.
- **Operational baseline:** CivicConnect targets proportionate, low-cost and best-effort availability for its initial low-to-medium workload. The system does not currently require enterprise high-availability architecture or horizontal scaling.

15.3 Open / Deferred Decisions

The following deployment and operational decisions remain deliberately deferred until sufficient M3 implementation and staging evidence is available:

- **Production application hosting provider:** Render and Railway remain candidate managed PaaS providers. The final production host will only be selected after validating Node.js resource usage, PostgreSQL connectivity, environment-variable and secret handling, availability behaviour, operational cost and compatibility with the selected production email-delivery approach.
- **Managed PostgreSQL provider:** PostgreSQL has already been selected as the database technology, but the final managed provider remains deferred. Neon and Supabase remain candidates and must be evaluated against connection pooling, storage limits, availability, backup/recovery capability, cost and the project's current recovery targets before final selection.
- **Attachment / image storage mechanism:** FR-106 requires durable attachment storage. Local application-instance storage is not considered suitable for production because files may be lost during restart or redeployment. The final durable storage mechanism or provider remains deferred until representative image-upload testing confirms storage requirements, file-size limits, performance impact, access control, backup/retention and cost.
- **Production email-delivery configuration/provider:** Nodemailer using SMTP remains the selected application-level integration mechanism and Ethereal remains suitable for development/testing. The final production SMTP provider/configuration remains deferred because it must be compatible with the selected production hosting environment and its outbound-network restrictions.
- **Production environment variables and secrets configuration:** Environment-specific configuration and secrets will remain external to source control. Final production values and provider-specific secret configuration will only be established once the production hosting, database and supporting services have been selected.
- **Automated production deployment:** GitHub Actions currently provides CI and verification, while production Continuous Deployment remains deferred. The team will first validate the integrated application in the dedicated local staging environment and establish a successful manual production deployment before automating deployment to the live environment.
- **Production monitoring and observability:** The current /health endpoint provides initial health verification, but the final logging, monitoring, uptime and failure-alerting approach remains deferred until the production hosting platform and its available operational tooling are selected.
- **Rollback and recovery validation:** Recovery targets and an initial recovery procedure have been identified, but the final production rollback and database-recovery process remains deferred until the selected hosting and database platforms can be tested through restore, redeployment and smoke-test evidence.

- **Horizontal scaling and external session storage:** The current deployment baseline remains a single application instance. Horizontal scaling and external session storage are deferred unless M3 workload or resource evidence demonstrates that the initial single-instance deployment is insufficient.

For detailed evidence requirements and expected finalisation points, refer to Section 10.5 – Deferred Deployment Decisions and Evidence Required.

References

Databricks (2026). *Concurrency control in DBMS: How locking, MVCC and optimistic strategies keep data consistent*. [online] Databricks. Available at: <https://www.databricks.com/blog/concurrency-control> [Accessed 27 Sept. 2026].

Douslimi, Y. (2021). *You shouldn't store your images/videos in a database*. [online] ENSIAS IT. Available at: <https://medium.com/ensias-it/you-shouldnt-store-your-images-videos-in-a-database-6a78ffa277b2> [Accessed 28 Sept. 2026].

Ethereal, 2026 *Ethereal Email – Fake SMTP Service for Email Testing*. [Online] Available at: <https://ethereal.email/> [Accessed 30 September 2026].

Express.js, 2026 *Express – Node.js web application framework*. [Online] Available at: <https://expressjs.com/en/> [Accessed 30 September 2026].

GitHub. “About Issues - GitHub Docs.” *GitHub Docs*, 2025, <https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>. Accessed 7 Sept. 2026.

GitHub, 2026 *GitHub Actions billing*. [Online] Available at: <https://docs.github.com/en/billing/concepts/product-billing/github-actions> [Accessed 30 September 2026].

GitHub. “Viewing Branches in Your Repository - GitHub Docs.” *GitHub Docs*, 2026, <https://docs-internal.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-branches-in-your-repository/viewing-branches-in-your-repository>. Accessed 7 Sept. 2026.

Google, 2026-a. *Google Workspace pricing*. [Online] Available at: https://workspace.google.com/pricing?hl=en_zh [Accessed 30 September 2026].

Google, 2026-b. *Limits for sending and receiving emails*. [Online] Available at: <https://support.google.com/mail/answer/22839> [Accessed 30 September 2026].

MailChannels (2025). *Why gmail sends your transactional emails to spam (and how to fix it)*. [online] MailChannels Blog. Available at: <https://blog.mailchannels.com/why-gmail-sends-your-transactional-emails-to-spam-and-how-to-fix-it/> [Accessed 25 Sept. 2026].

Microsoft (2026). *Emails landing at others spam - Microsoft Q&A*. [online] MicrosoftLearn. Available at: <https://learn.microsoft.com/en-au/answers/questions/5825500/emails-landing-at-others-spam> [Accessed 25 Sept. 2026].

Neon, 2025. *Neon's New Pricing, Explained: Usage-Based, No Minimum*. [Online] Available at: <https://neon.com/blog/new-usage-based-pricing> [Accessed 30 September 2026].

Node.js, 2026 *Node.js Releases*. [Online] Available at: <https://nodejs.org/en/about/previous-releases> [Accessed 30 September 2026].

Nodemailer, 2026 *License*. [Online] Available at: <https://nodemailer.com/license> [Accessed 30 September 2026].

Nodemailer (2025). *SMTP transport | nodemailer*. [online] Nodemailer.com. Available at: <https://nodemailer.com/smtp> [Accessed 25 Sept. 2026].

Patel, K. (2026). *Nodemailer NPM: A security review and safe usage guide*. [online] Safeguard. Available at: <https://safeguard.sh/resources/blog/nodemailer-npm> [Accessed 25 Sept. 2026].

PostgreSQL Global Development Group, 2026 *PostgreSQL Licence*. [Online] Available at: <https://www.postgresql.org/about/licence/> [Accessed 30 September 2026].

Railway, 2026-a. *Pricing Plans*. [Online] Available at: <https://docs.railway.com/pricing/plans> [Accessed 30 September 2026].

Railway, 2026-b. *Networking*. [Online] Available at: <https://docs.railway.com/networking> [Accessed 30 September 2026].

Railway, 2026-c. *Outbound Networking*. [Online] Available at: <https://docs.railway.com/networking/outbound-networking> [Accessed 30 September 2026].

Railway, 2026-d. *Restart Policy*. [Online] Available at: <https://docs.railway.com/deployments/restart-policy> [Accessed 30 September 2026].

Render, 2026-a. *Deploy for Free*. [Online] Available at: <https://render.com/docs/free> [Accessed 30 September 2026].

Render, 2026-b. *Pricing*. [Online] Available at: <https://render.com/pricing> [Accessed 30 September 2026].

Schuman, Evan. "AI Token Freeloaders Are Coming for Your Customer Support Chatbot." *CIO*, 8 Apr. 2026, <https://www.cio.com/article/4155404/ai-token-freeloaders-are-coming-for-your-customer-support-chatbot.html>. Accessed 7 Sept. 2026.

Supabase, 2026a. *About billing on Supabase*. [Online] Available at: <https://supabase.com/docs/guides/platform/billing-on-supabase> [Accessed 30 September 2026].

Supabase, 2026b. *Project Pausing*. [Online] Available at: <https://supabase.com/docs/guides/platform/free-project-pausing> [Accessed 30 September 2026].

Supabase, 2026-a. *Pricing & Fees*. [Online] Available at: <https://supabase.com/pricing> [Accessed 30 September 2026].

Baseline Sign off

Appendix D — Baseline Sign-Off Template

Project	CivicConnect
Baseline Type	PED Milestone 1
Version	Version1
Date	9 September 2026
Scope reviewed	☒ YES ☐ NO
Requirements/traceability checked	☒ YES ☐ NO
Risk review completed	☒ YES ☐ NO
Repository/governance controls checked	☒ YES ☐ NO
Outcome	☒ ACCEPTED ☐ CONDITIONALLY ACCEPTED / REVISION REQUIRED